Newcastle University

MAS8391 MMathStat Project

2023/24

Dynamic rating systems for professional golfers

Author:
Yusuf Latif

Supervisor:
Dr. Daniel Henderson

April 26, 2024



Contents

1 Introduction **3**
- 1.1 Elo-rating . . . 3
- 1.2 Notation . . . 4
- 1.3 Powell's generalized Elo-rating system . . . 4

2 Calculating Probabilities and Updating β **5**
- 2.1 Generalised Plackett-Luce model . . . 5
- 2.2 GPL probability equation . . . 6
- 2.3 Deriving β's update equation . . . 6

3 Initial Testing **9**
- 3.1 Simple Example . . . 9
- 3.2 Test on simulated data . . . 10
- 3.3 Applying to Real Data . . . 11
- 3.4 Log Score . . . 15

4 Updating at different times **20**
- 4.1 Updating worst to best . . . 20
- 4.2 Updating after each stage . . . 22

5 Competitor specific learning rates, γ_k **24**
- 5.1 Exponential learning rate update . . . 24
- 5.2 Finding the optimal values for the exponential update equation . . . 24
- 5.3 Exponential learning rate update results . . . 26
- 5.4 Powell's competitor specific learning rate . . . 29
- 5.5 Deriving negative second derivative of $\log(P^{(j)})$ . . . 29
- 5.6 Reciprocal update equation for γ_k . . . 30
- 5.7 Finding the optimal values for the second derivative update . . . 32
- 5.8 Log second derivative update results . . . 33
- 5.9 Which competitor specific learning rate update is best for Golf? . . . 35

6 Conclusion **36**
- 6.1 Final Results . . . 36
- 6.2 What Next? . . . 38
- 6.3 Acknowledgements . . . 38

A Final function for rating update algorithm, in R **39**

Abstract

The aim of this project is to develop a rating system, for professional golfers, capable of accurately predicting the outcomes of golf competitions by using the ratings assigned to each individual golfer. The rating, for each golfer, should then update after every round, based on their performance in the round, their initial rating score, as well as the initial ratings and performance of their competitors. This information can be used, not only to identify the best and worst performing golfers, but also to assist with calculating betting odds. Although there is a rating system for professional golfers[1], it cannot be used to predict future competition outcomes, this project addresses the need for a reliable method to rank players, based on performance, and predict competition outcomes.

Chapter 1

Introduction

1.1 Elo-rating

The Elo-rating system was developed by Arpad Elo, in the 1950s [5], as a way to rank chess players and was officially adopted by the U.S. Chess Federation in 1960 as well as by FIDE in 1970 [3]. Although it was initially used for chess, it can be used in any 2 player competitions to effectively rank competitors, where the higher their rating, β, the more skilled the player is considered to be. There are many variations of the Elo-rating system today, with some being used to rank competitors in competitions with more than 2 competitors. The traditional Elo-rating system rates competitors in the following way. If we have two competitors i and j, who have ratings of β_i and β_j respectively, then the probability that competitor i beats j is

$$\mathbb{E}[X(i\,beats\,j)] = \frac{e^{\beta_i}}{e^{\beta_i} + e^{\beta_j}}, \tag{1.1}$$

where $X(i\,beats\,j)$ is a binary random variable which equals 1 if i beats j and 0 otherwise. This equation comes from the Bradley Terry model [7].
After the two competitors have completed their match, we would then update both their scores using the following equation

$$\beta_i \leftarrow \beta_i + \gamma(X(i\,beats\,j) - \mathbb{E}[X(i\,beats\,j)]), \tag{1.2}$$

where γ is a positive constant and $X(i\,beats\,j)$ is 1 if i did win and 0 otherwise.
Note that $X(i\,beats\,j) - \mathbb{E}[X(i\,beats\,j)]$ is the derivative of the $\log(\mathbb{E}[X(i\,beats\,j)])$ with respect to β_i. Thus, the Elo-rating system can be seen as applying stochastic gradient ascent [8] to the log likelihood.

The Elo-rating system works very well for one-on-one competitions and effectively updates each competitors rating. We can see from (1.2) that, if i beats j, then $X(i\,beats\,j) - \mathbb{E}[X(i\,beats\,j)]$ is positive, so β_i will increase. However, as the probability $0 < \mathbb{E}[X(i\,beats\,j)] < 1$, if i is very likely to beat j then β_i will only increase a little if i wins, but decrease a lot if i loses. This rewards competitors more for beating opponents more skilled than they are and vice versa.

These ratings also scale well after a large number of matches and with a large number of competitors as only the ratings of the competitors competing in a match update after the match. We would like to develop an Elo-rating system for golfers based on their position in a round of a given competition.

1.2 Notation

Before continuing we must establish some basic notation which will later be expanded on.

If we have a list of competitors, $\mathcal{K}$, of length K, then y is the ordered set of competitors in $\mathcal{K}$ and s is the set positions in y. For all $i \in \mathcal{K}$, β_i represents competitor i's rating score.

Example: Let $K = 5$, with the competitors labelled $\mathcal{K} = \{1, 2, 3, 4, 5\}$, then if $y = \{1, 2, 4, 5, 3\}$ and $s = \{1, 2, 3, 3, 4\}$. This shows us that competitor 1 came 1st, and competitor 2 came 2nd, but competitors 4 and 5 drew in 3rd place. Then, competitor 3 came last.

1.3 Powell's generalized Elo-rating system

Although the Elo-rating system works well for one-on-one competitions, if there are multiple competitors, then how do you update the competitors which come 2nd or 6th, or competitors that draw in 4th?

Powell [10] ran into a similar problem when he was making a generalized Elo-rating system for races. To get around the problem of updating multiple competitors rating, he suggested splitting the event where y and s occur into stages. The 1st stage would focus on last place. All competitors that lost at that stage, so they came last place, their β's will decrease. On the other hand, the competitors that won in that stage, so they didn't come last, their β's will increase. Then, the 2nd stage would focus on second last place. One would repeat this process again but excluding the competitors who lost in the previous stage. The last stage would only involve the competitors who came 1st and 2nd.

Example: If we continue from the previous example where there are $K = 5$ competitors labelled $\mathcal{K} = \{1, 2, 3, 4, 5\}$, where $y = \{1, 2, 4, 5, 3\}$ and $s = \{1, 2, 3, 3, 4\}$, then the β updates would go as follows:

1st stage: competitor $\{3\}$ placed 4th, so $\{\beta_3\}$ decreases and $\{\beta_1, \beta_2, \beta_4, \beta_5\}$ increase.

2nd stage: remove competitor $\{3\}$, competitors $\{4, 5\}$ placed 3rd, so $\{\beta_4, \beta_5\}$ decrease and $\{\beta_1, \beta_2\}$ increase.

3rd stage: remove competitors $\{4, 5\}$, competitor $\{2\}$ placed 2nd, so $\{\beta_2\}$ decreases and $\{\beta_1\}$ increases.

Note: Another stage is not needed as the probability for that stage is 1, which we will see later. Therefore, there is $max(s) - 1$ stages, where $max(s)$ is the largest number in s, or the last position.

In the next chapter, we will investigate how to develop a similar rating system for golfers.

Chapter 2

Calculating Probabilities and Updating β

Before we can implement this method of updating β_i's, we need to find a suitable model to calculate the probability that the event y and s occurs. For Powell's [10] rating system, he used a generalised Plackett-Luce model, which does not allow ties, whereas ties in golf are commonplace. Therefore we will use a generalized Plackett-luce model, which allows for ties in ordered data.

2.1 Generalised Plackett-Luce model

Let $c \in \mathcal{K}$, where $\mathcal{K}$ is the set of length K, y be the rank ordered set of $\mathcal{K}$ and $\lambda_c > 0$ the strength parameter for c. Then under the Plackett-Luce model [9], the probability of observing y is

$$Pr(Y = y \mid \lambda) = \prod_{i=1}^{K-1} \frac{\lambda_{y_i}}{\sum_{l=i}^{K} \lambda_{y_\ell}}, \tag{2.1}$$

The Plackett-Luce model has a generative representation in terms of ordering independent exponential latent variables. Specifically, for the Plackett-Luce model, let $W_c \sim Exp(\lambda_c)$, $c \in \mathcal{K}$, be independent exponential random variables. Then, it can be shown that

$$Pr(W_{y_1} < W_{y_2} < ... < W_{y_K} \mid \lambda) = \prod_{j=1}^{K-1} \frac{\lambda_{y_j}}{\sum_{l=j}^{K} \lambda_{y_\ell}}, \tag{2.2}$$

which is precisely the Plackett-Luce probability shown in Equation (2.1).

However, we can't find the probability of a tie with an $W_c \sim Exp(\lambda_c)$. Therefore, it makes sense to use a geometric distribution instead of an exponential, $W_c \sim Geom(\theta_c)$ instead of $W_c \sim Exp(\lambda_c)$, where θ_c is the strength parameter of $c \in \mathcal{K}$, where the probability mass function is

$$Pr(W = w) = (1 - \theta_k)^{w-1}\theta_k,\ w \in \{1, 2, 3...\}$$

This generalised Plackett-Luce model (GPL) is what we will use to calculate the probability of any event y and s occurring, but we need to calculate the probability equation.

2.2 GPL probability equation

Using this generalized Plackett-Luce model, we obtain a new probability equation of observing y and s. If there are n stages, so n is the last number in s, the probability of y and s occurring is $P = P^{(1)}P^{(2)}...P^{(n-1)}$, where $P^{(i)}$ is the probability that the competitors in y, which have not won a stage yet, win at stage i. For an example,

$$P^{(1)} = \frac{\theta_k \prod_{i \neq \{k\}}(1-\theta_i)}{1-(1-\theta_k)\prod_{i\neq\{k\}}(1-\theta_i)} \tag{2.3}$$

is the probability competitor k comes 1st and

$$P^{(2)} = \frac{\theta_\ell \prod_{i \neq \{k,\ell\}}(1-\theta_i)}{1-(1-\theta_\ell)\prod_{i\neq\{k,\ell\}}(1-\theta_i)} \tag{2.4}$$

is the probability competitor ℓ comes 2nd, given that k has come 1st. You can see that, if competitor q came 3rd, $P^{(3)}$ would be in the same form as $P^{(2)}$ but excluding $\{k, \ell, q\}$ from both $\prod$. The GPL model can also give the probability that 2 or more competitors tie at a given stage, as the next example illustrates.

Example: If we continue the example from Section 1.2 and 1.3, but remove the competitor who came 1st, so $y = \{2, 4, 5, 3\}$ and $s = \{1, 2, 2, 3\}$. Equation (2.5) shows the probability that events y and s occur.

$$P = P^{(1)}P^{(2)} = \frac{\theta_2(1-\theta_4)(1-\theta_5)(1-\theta_3)}{1-((1-\theta_2)(1-\theta_4)(1-\theta_5)(1-\theta_3))} \frac{\theta_4\theta_5(1-\theta_3)}{1-((1-\theta_4)(1-\theta_5)(1-\theta_3))}, \tag{2.5}$$

where θ_i is the strength parameter of competitor i. Note that at stage 2, competitors 4 and 5 tie and so we get $\theta_4\theta_5$ in the numerator of the expression for $P^{(2)}$.

2.3 Deriving β's update equation

For the general elo-rating system, they update each competitor's rating after every match with the expression (1.2), from Section 1.1, where $(X(i \text{ beats } j) - \mathbb{E}[X(i \text{ beats } j)])$ is the derivative of the log probability that i beats j, with respect to β_i. We will be updating competitor's β values in a similar way, but updating after each stage, instead of a whole game/round. Therefore we need to first find the log of $P^{(j)}$, where $j \in s$ represents the stage. If we let competitor k be come 1st and competitors l and q draw 2nd place, then

$$\log(P^{(1)}) = \log(\theta_k) + \sum_{i\neq\{k\}} \log(1-\theta_i) - \log(1-\prod(1-\theta_i)) \tag{2.6}$$

$$\log(P^{(2)}) = \log(\theta_l) + \log(\theta_q) + \sum_{i\neq\{k,l,q\}} \log(1-\theta_i) - \log(1-\prod_{i\neq\{k\}}(1-\theta_i)) \tag{2.7}$$

The remaining $log(P^{(j)})$'s follow this pattern, removing the winners of stage j from the next stage, $j+1$. We can see that the general form of these equations remain the same.

Let $\beta_k = logit(\theta_k) = \log(\theta_k) - \log(1-\theta_k)$, therefore

$$\theta_k = \frac{e^{\beta_k}}{1+e^{\beta_k}}, \text{ therefore } \log(\theta_k) = -log(1+e^{\beta_k})$$

$$\text{and } 1-\theta_k = \frac{e^{-\beta_k}}{1+e^{-\beta_k}}, \text{ therefore } \log(1-\theta_k) = -\log(1+e^{-\beta_k})$$

We will need the following to help calculate the log likelihood derivative

$$\frac{\partial \log(\theta_k)}{\partial \beta_k} = \frac{e^{-\beta_k}}{1+e^{-\beta_k}} = 1-\theta_k \tag{2.8}$$

$$\frac{\partial \log(1-\theta_k)}{\partial \beta_k} = -\frac{e^{\beta_k}}{1+e^{\beta_k}} = 1-\theta_k \tag{2.9}$$

$$\frac{\partial (1-\theta_k)}{\partial \beta_k} = \frac{\partial (1+e^{\beta_k})^{-1}}{\partial \beta_k} = -\frac{e^{\beta_k}}{(1+e^{\beta_k})^2} = -\theta_k(1-\theta_k) \tag{2.10}$$

Now we can differentiate $log(P^{(j)})$ with respect to β_k. If $y = \{k, l, q, ...\}$ and $s = \{1, 2, 2, 3, ..., n\}$, so that competitor k came 1st and competitors l and q draw in 2nd. Then, for the 1st stage...

$$\frac{\partial \log(P^{(1)})}{\partial \beta_k} = (1-\theta_k) - \frac{\{\theta_k(1-\theta_k)\}\prod_{i\neq\{k\}}(1-\theta_i)}{1-\prod(1-\theta_i)} \tag{2.11}$$

This can be simplified to

$$\frac{\partial \log(P^{(1)})}{\partial \beta_k} = (1-\theta_k) - (1-\theta_k)P^{(1)} = (1-\theta_k)(1-P^{(1)}) \tag{2.12}$$

This can be written as

$$\frac{\partial \log(P^{(1)})}{\partial \beta_k} = (1-P_k^{(1)}) - \theta_k(1-P_k^{(1)}) \tag{2.13}$$

where $P_i^{(j)}$ is the probability competitor i wins at stage j, for $i \in y$ and $j \in s$.

Now if we differentiate with respect to β_l we get the following

$$\frac{\partial \log(P^{(1)})}{\partial \beta_l} = -\theta_l - \frac{\{\theta_l(1-\theta_l)\}\prod_{i\neq\{k,l\}}(1-\theta_i)}{1-\prod(1-\theta_i)} \tag{2.14}$$

This can be simplified to

$$\frac{\partial \log(P^{(1)})}{\partial \beta_l} = (0-P_l^{(1)}) - \theta_l(1-P_l^{(1)}) \tag{2.15}$$

Similarly, when we differentiate with respect to β_q, for $q \neq k$, we get

$$\frac{\partial log(P^{(1)})}{\partial \beta_q} = (0-P_q^{(1)}) - \theta_q(1-P_q^{(1)}) \tag{2.16}$$

Continuing this process for the 2nd stage, we get

$$\frac{\partial \log(P^{(2)})}{\partial \beta_k} = 0 \tag{2.17}$$

as competitor k won in the previous stage. When differentiating by β_l, we get

$$\frac{\partial \log(P^{(2)})}{\partial \beta_l} = (1-\theta_l) - \frac{\{\theta_l(1-\theta_l)\}\prod_{i\neq\{k,l\}}(1-\theta_i)}{1-\prod(1-\theta_i)} \tag{2.18}$$

This simplifies to

$$\frac{\partial \log(P^{(2)})}{\partial \beta_l} = (1-P_l^{(2)}) - \theta_l(1-P_l^{(2)}) \tag{2.19}$$

Similarly, when we differentiate with respect to β_q, we get

$$\frac{\partial \log(P^{(2)})}{\partial \beta_q} = (1-P_q^{(2)}) - \theta_q(1-P_q^{(2)}) \tag{2.20}$$

If we continue this process through the rest of stages, we get the following general form for the log likelihood derivative

$$\frac{\partial \log(P^{(j)})}{\partial \beta_k} = (\mathbf{I}_k^{(j)} - P_k^{(j)}) - \theta_k(1-P_k^{(j)}), \tag{2.21}$$

where j represents the stage, k is the competitor, $P_k^{(j)}$ is the probability that k wins at stage j and $\mathbf{I}_k^{(j)} = 1$ if competitor k wins at stage j and 0 otherwise.

Now that we have a general form for the log likelihood derivative, we can use use gradient ascent[8] to form an update equation for a competitors rating, β

$$\beta_k \leftarrow \beta_k + \gamma((\mathbf{I}_k^{(j)} - P_k^{(j)}) - \theta_k(1-P_k^{(j)})), \tag{2.22}$$

where $\gamma > 0$ is some constant, known as the learning rate, k is a competitor involved in that stage and β_k is their rating.

We can see that if competitor k loses at stage j, their rating will decrease, but if k wins, their rating will increase. Also, the lower k's probability to win the more their rating will increase if they do win. On the other hand, the lower k's probability to win the less their rating will decrease if k loses. This would work the opposite way if k had a high probability of winning at stage j.

Chapter 3

Initial Testing

3.1 Simple Example

Now that we have an algorithm to update the ratings, β, we can test its effectiveness on a small scale. Assume there are $K = 5$ competitors, where $\mathcal{K} = \{1, 2, 3, 4, 5\}$, which play a round of golf. For this example, we will use a learning rate of $\gamma = 0.1$. Therefore, Equation (3.1) is the update equation for this example is.

$$\beta_k \leftarrow \beta_k + 0.1((\mathbf{I}_k^{(j)} - P_k^{(j)}) - \theta_k(1 - P_k^{(j)})), \tag{3.1}$$

where $k \in K$ and $j \in s$.

Before the game, the competitors had rating $\beta = \{\beta_1, \beta_2, \beta_3, \beta_4, \beta_5\} = \{-1, -1.2, -1.4, -1.6, -1.8\}$, therefore, we would expect competitor 1 to come 1st and 5 to come 5th as they have the largest and smallest ratings, respectively. The results of the round are such that $y = \{1, 2, 4, 5, 3\}$ and $s = \{1, 2, 3, 3, 4\}$. Using this update equation, we get the following new rating scores for each competitor.

Table 3.1

Competitors	1	2	3	4	5
Initial $\beta's(3.d.p)$	-1.000	-1.200	-1.400	-1.600	-1.800
Final $\beta's(3.d.p)$	-0.940	-1.175	-1.511	-1.594	-1.779
Change in $\beta's(3.d.p)$	0.060	0.025	-0.111	0.006	0.021

The updated β values are as expected, with β_1 and β_2 both increasing, β_3 decreases a lot and β_4 and β_5 slightly increase. Even though competitors 4 and 5 drew in 3rd place, competitor 5's rating score increased more as they were most likely to come last, according to their rating.

3.2 Test on simulated data

Before we apply this rating system to the real data, it can be useful to test it on simulated data first. We will do this by using the uniform distribution, $U(0, 1)$, to generate a unique true θ for each competitor, θ_T. Then using the geometric distribution, we can simulate results for a competition, using the competitors θ_T as the probabilities. After simulating n competitions we can set a predictive θ, θ_P, to each competitor. Then, based on the results generated using the geometric distribution, we can update these predictive θ's using equation (2.22). After performing all updates, the predictive θ for each competitor should be similar to the true θ generated using the uniform distribution, e.g. $\theta_P \approx \theta_T$.

We will set the initial predictive θ's to 0, simulating 200 competitions for 20 competitors, with a learning rate of $\gamma = 0.2$. Figure 3.1 shows us the results.

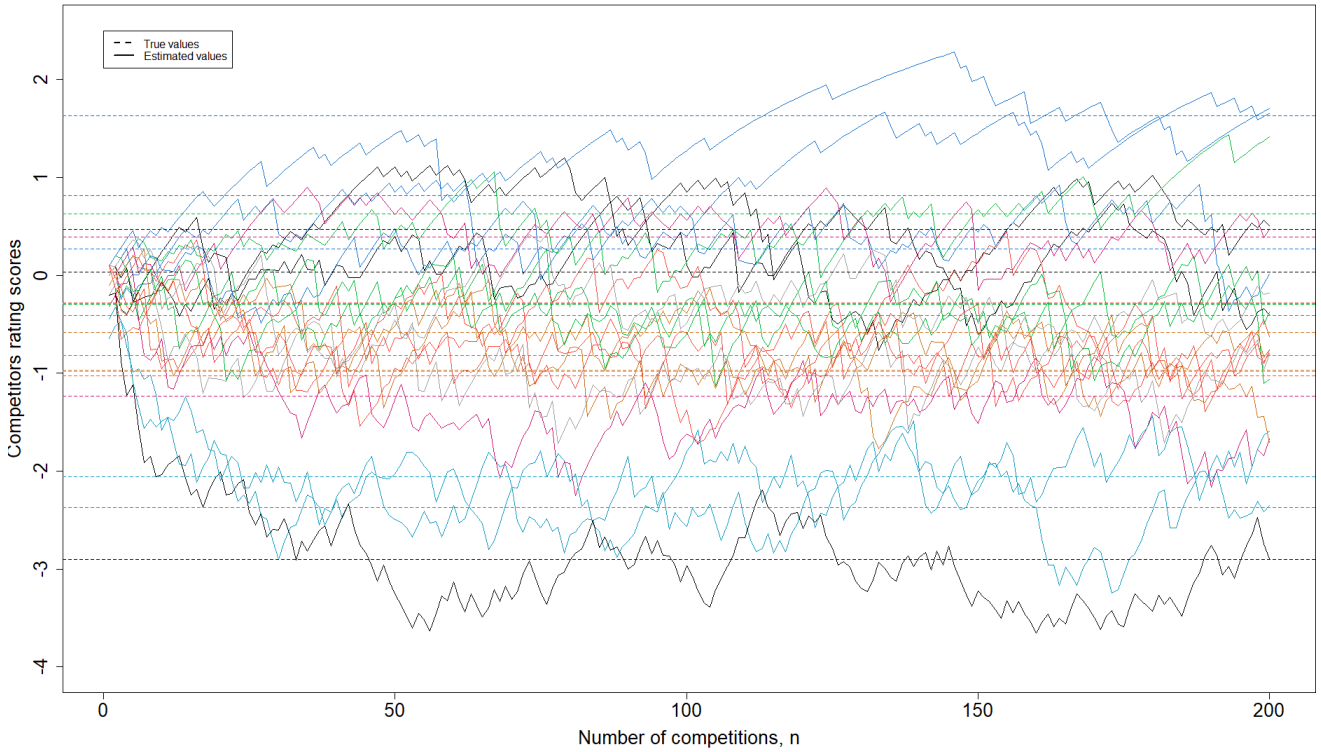


Figure 3.1: Simulation of Elo-rating system updates

It seems to have worked well. All the ratings reached and remained around their predicted true value. We can see that the worse ratings take around 50 updates before they reach their predicted true value, whereas the better ratings take around 75 updates. We also see that the worse ratings predicted true values are further away from the initial rating, 0, compared to the better ratings which are closer to 0. This may be because the worse ratings are involved in more stages, therefore receiving larger updates.

3.3 Applying to Real Data

Before we test this rating system on the real data, we need to discuss the layout of the data-set itself. The data-set we will be using is data of round by round scores in tournaments for male professional golfers from 2018-2022, provided by Smartodds[2]. Smartodds is a company which provides statistical research and consulting services about sports. With the vast amount of data they have on various sports, they develop and implement predictive statistical models for sporting events In the data-set, each competition is split into 4 rounds, where each round is a golf match. For some competitions, after the first 2 rounds, they eliminate the bottom half of competitors from that competition. Therefore, it makes the most sense to update competitors ratings, β, after every round instead of every competition. There are 1647 golfers and 398 competitions, so 1592 rounds, in this data-set which will result in 1592 updates.

After cleaning up the data and re-formatting it, we are ready for the first test. We will set the initial rating, β_0, to -2 for all competitors and the learning rate, γ, to 0.1, giving Figure 3.2.

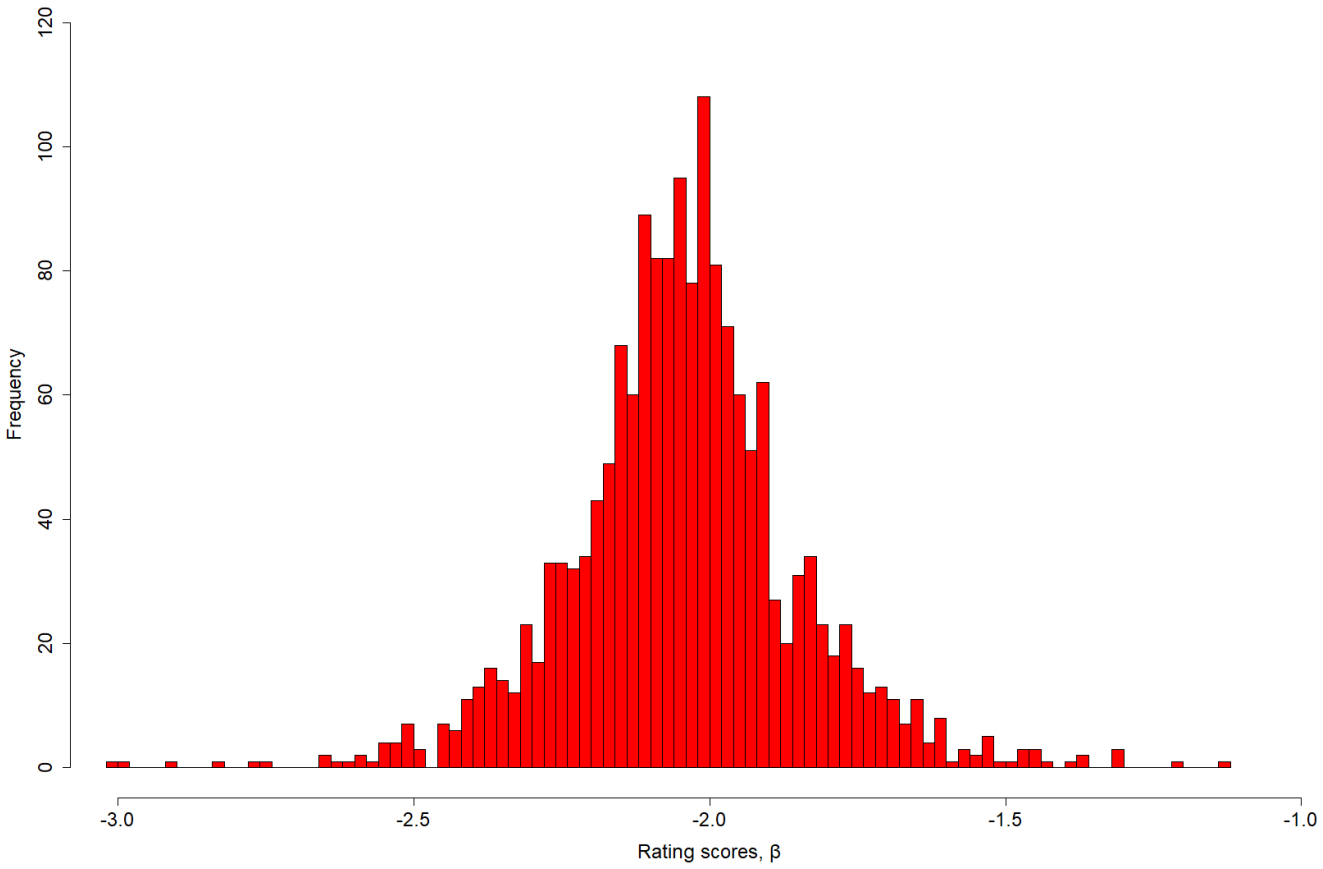


Figure 3.2: Histogram of the final rating scores, β

These results seem good, as we have a mix of strong and weak competitors as well as a mode around the initial rating, $\beta_0 = -2$, and a mean of -2.04. It almost resembles a normal distribution but its a bit too peaked and there are more weak competitors than strong, according to the rating system.

We can also plot each competitors path to their final rating score in Figure 3.3.

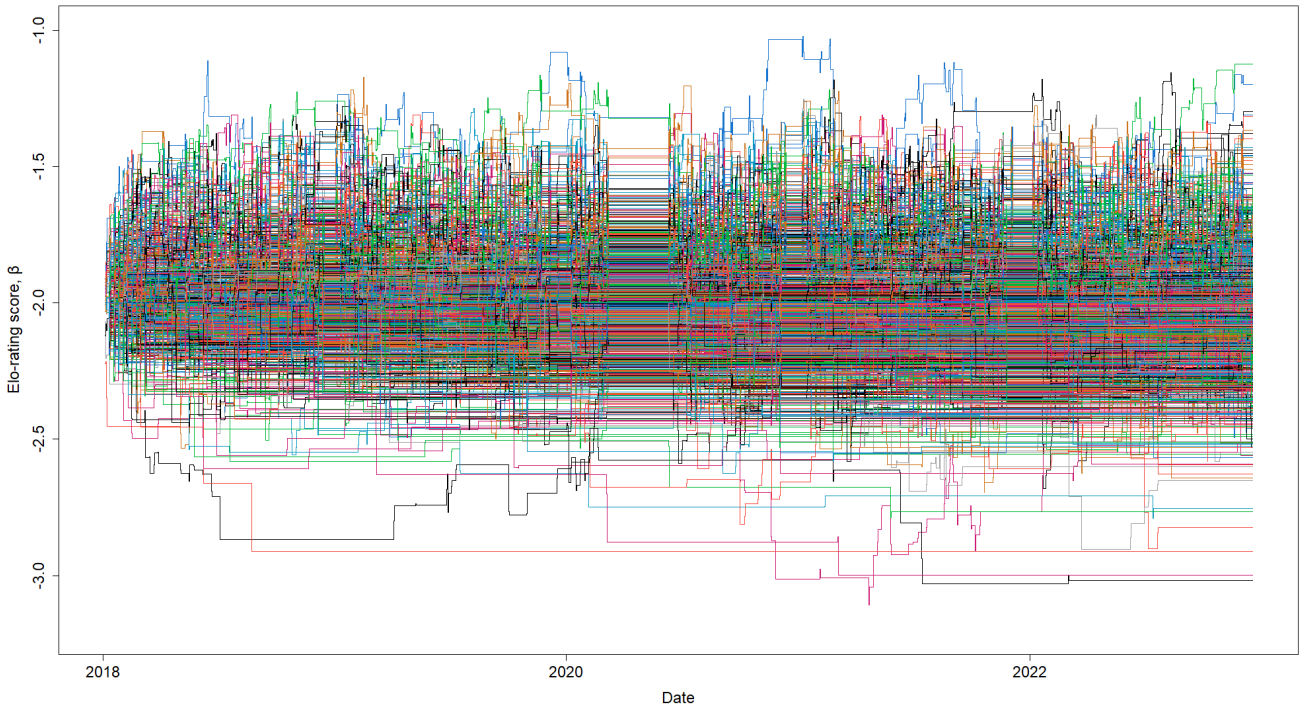


Figure 3.3: Histogram of final rating scores, β

Although we can see events such as every competitors inactivity during the COVID-19 pandemic, this plot can be overwhelming and it is hard to follow a competitor's rating path. Therefore it will be better to concentrate on some specific competitors. Table 3.1 shows the final ratings for the top and bottom 5 competitors.

Table 3.2

Top 5		Bottom 5	
Player	β	Player	β
Rory McIlroy	-1.126	Smylie Kaufman	-3.017
Jon Rahm	-1.200	Robert Gamez	-2.998
Tony Finau	-1.300	Petr Dedek	-2.912
Xander Schauffele	-1.312	Matthew Every	-2.824
Scottie Scheffler	-1.317	Keith Clearwater	-2.765

Similarly to our simulation test results, we can see the strongest competitors have moved less than 0.9 away from the initial rating, $\beta_0 = -2$, whereas some of the weakest competitors have moved a little further, with the weakest competitor, Smylie Kaufman, moving more than 1 away from the initial rating. This may be because competitors who come last in a round are involved in the most stages, therefore receive a larger update.

Figure 3.4 displays ratings for the top and bottom 5 competitor's ratings throughout 2018 to the end of 2022.

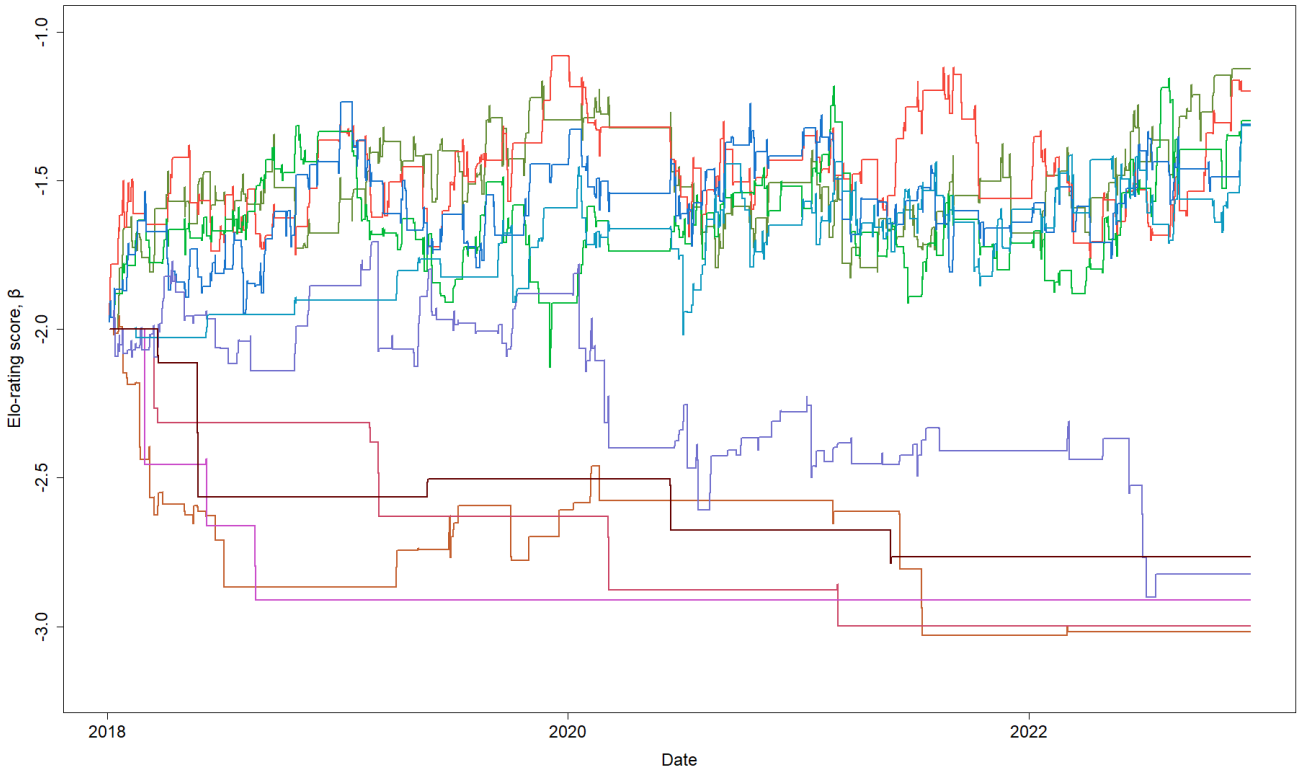


Figure 3.4: Top and bottom 5 golf competitors, based on their final ratings, β

This plot seems to work well, with the top 5 best competitors having a much higher rating than the bottom 5. It is also interesting to see events such as the COVID-19 pandemic, where all competitions seemed to stop, resulting in no updates for around 4 months. For active competitors, we can see it takes around 6 months for their ratings to become trapped in a certain range of ratings. The top 5 seem to fluctuate between -1.75 and -1.25. Similarly, for the bottom 5, once they fall below -2.5 they seem to struggle to increase their rating, or they stop competing.

Matthew Every in particular, has a unique path compared to other competitors in the table above, as can be seen in Figure 3.5.

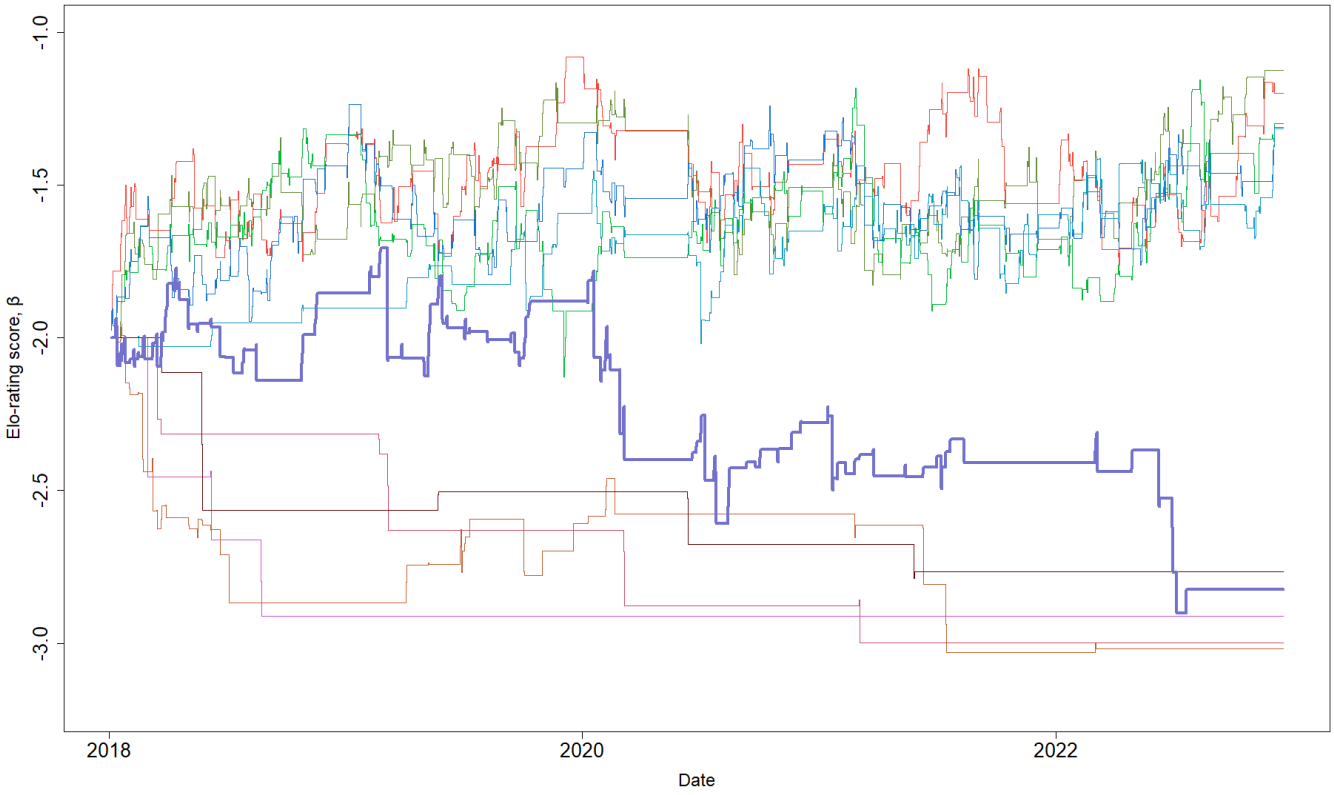


Figure 3.5: Matthew Every's rating score compared to other competitors

After the first 6 months, Matthew's rating score stayed in the range of -2.2 to -1.75, which we can see from Figure 3.1 is quite average. However, just before the pandemic of 2020, he seems to have performed poorly in a number of competitions. After the pandemic, his rating remained in-between -2.25 and -2.5 until summer 2022 where he again performs poorly a number of times, and seems to stop competing at a rating on -2.82. If we were to predict Matthew's final rating score based on his performance from 2018 to 2020, we would assume he would have remained in the range of -2.2 to -1.75. This is a reminder that, although the rating scores can predict outcomes of matches, there is a chance the unlikely happens.

3.4 Log Score

Even though our initial testing gives sensible results, we need to think about how we can improve our rating system. One improvement we can make is finding the optimal learning rate, γ, and initial rating, β_0. We can find the best value for both γ and β_0 by finding the largest log score.

The log score [6] is the sum of log probabilities of results that occurred, that is

$$S_L = \sum_{i=1}^{n} P, \tag{3.2}$$

where S_L is the log score, n is the number of competitions and P is the probability that the results that actually occurred do occur, calculated from the generalised Plackett-Luce model. The larger S_L, the more accurate the predictions are.

Using the log score, we can see which values for γ and β_0 will give the most accurate rating score for the competitors. As the learning rate, γ, has a larger impact on the rating scores, it makes the most sense to optimize it first, then optimize the initial rating, β_0. Figure 3.6 shows us the log scores for various different learning rates and initial ratings.

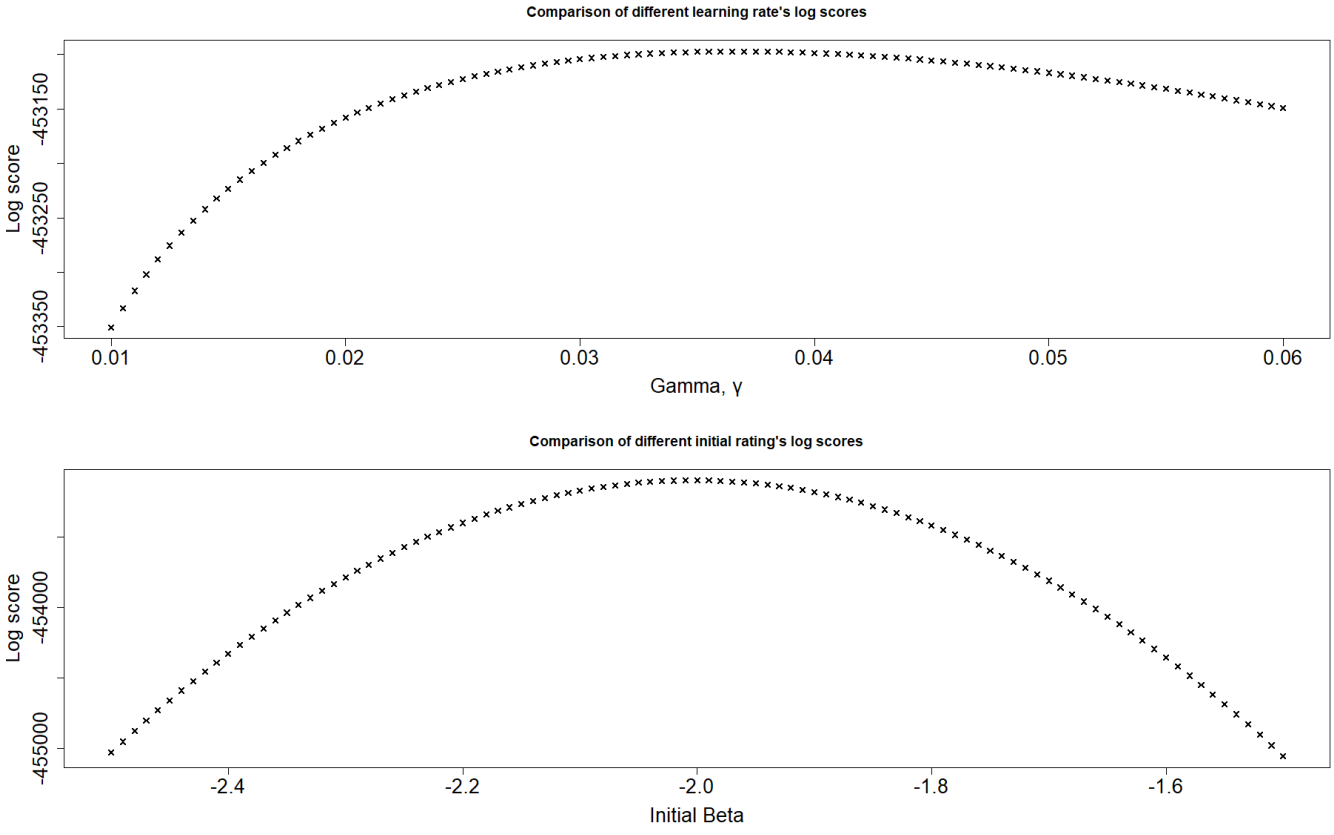


Figure 3.6: Finding the most optimal learning rate, γ, and initial rating score, β_0

I tested various γ values when $\beta_0 = -2$, comparing their log scores. The largest log score was -453097.1, when $\gamma = 0.037$. Then, I repeated this process, but for various β_0 values, when $\gamma = 0.037$. This gave us the same results, with the largest log score being -453097.1 when $\beta_0 = -2$.

We can now repeat the algorithm with the new optimal values, $\gamma = 0.037$ and $\beta_0 = -2$. This results in the histogram of final ratings, as shown in Figure 3.7.

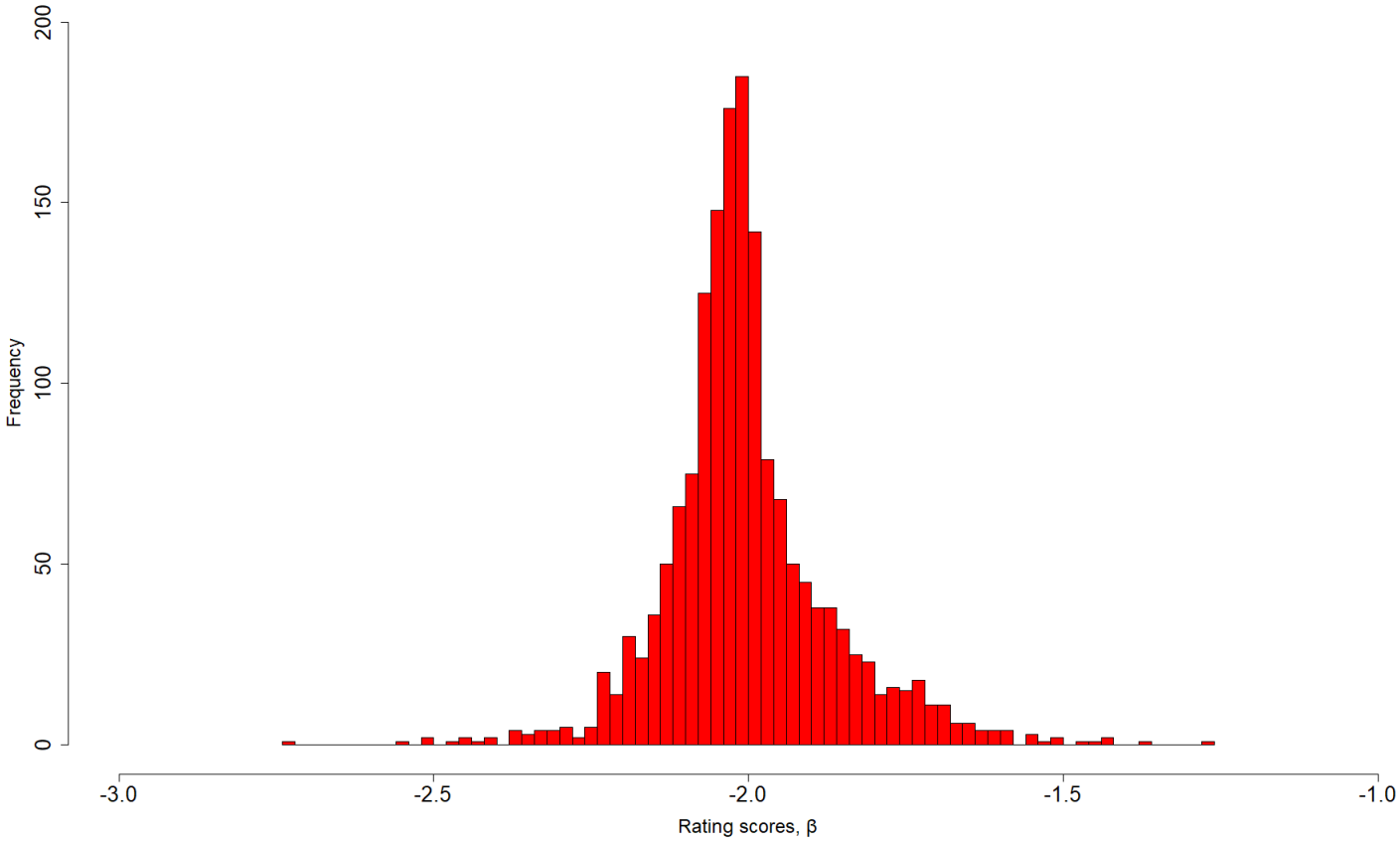


Figure 3.7: Histogram of the final rating scores, β

These results are similar to our original results, with most competitor's rating scores staying around the initial rating score, -2. However, it seems more concentrated around the initial rating, with far less competitors with rating scores outside the range $(-2.25, -1.75)$. This is most likely due to a smaller learning rate, resulting in smaller updates. The mean final rating score is -2.00(2DP).

Figure 3.8 plots the evolution of ratings over time,

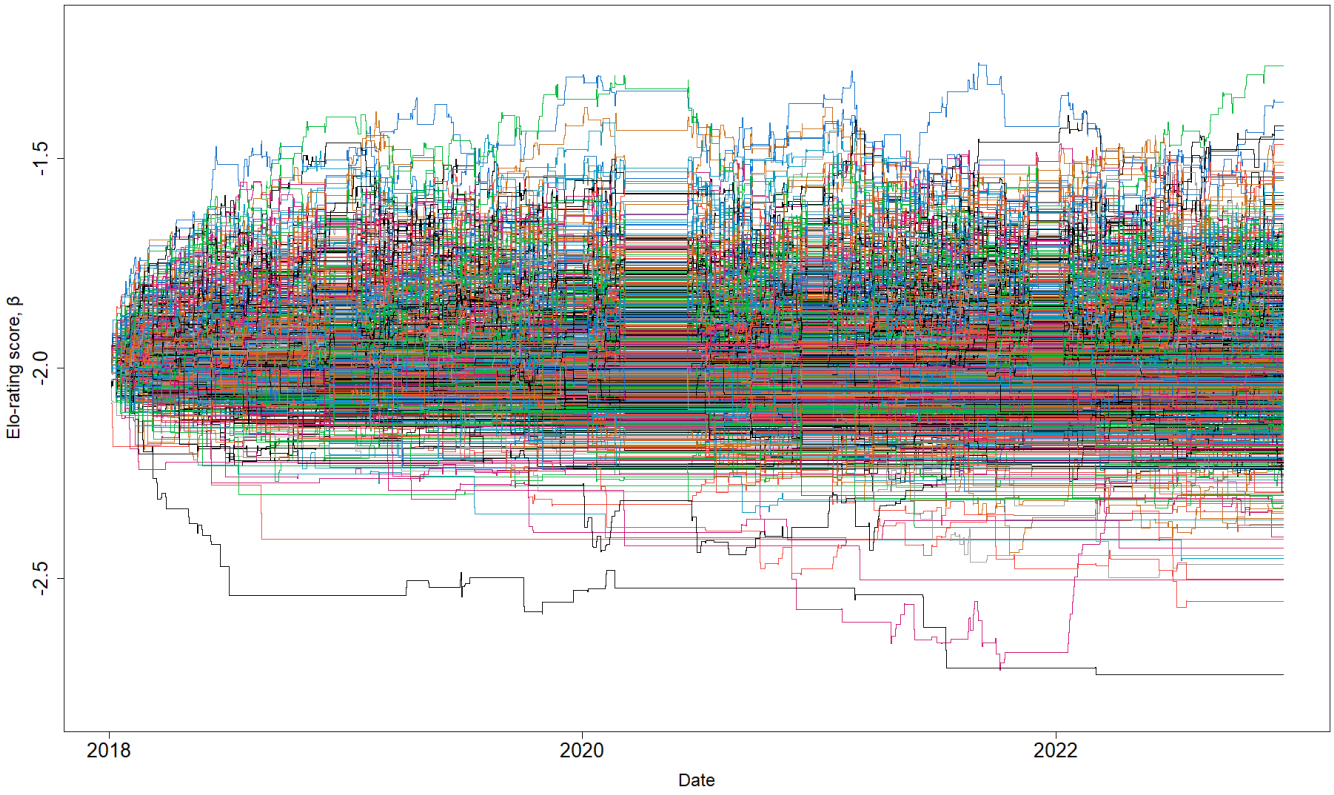


Figure 3.8: Rating scores, β, for all competitors

Again, the plot is hard to follow so we can focus on the 5 best and worst competitors. Table 3.3 shows us the final rating scores for the 5 best and worst competitors.

Table 3.3

Top 5		Bottom 5	
Player	β	Player	β
Rory McIlroy	-1.280	Smylie Kaufman	-2.728
Jon Rahm	-1.367	Matthew Every	-2.554
Tony Finau	-1.422	Robert Gamez	-2.504
Xander Schauffele	-1.433	Kevin Stadler	-2.501
Patrick Cantlay	-1.454	Parker McLachlin	-2.466

The top 5 competitors are the same as before, except Patrick Cantlay, as well as being in the same positions. Similarly, the bottom 5 competitors are mostly the same as our original test, except for Kevin Stadler and Parker McLachlin, and some positions have changed. However, the competitors which were in the top 5 best and worst but have now been replaced are still in similar positions, within the top 10 best and worst. So the results of both updates yield similar results.

Figure 3.9 shows the top 5 best and worst competitors rating scores path

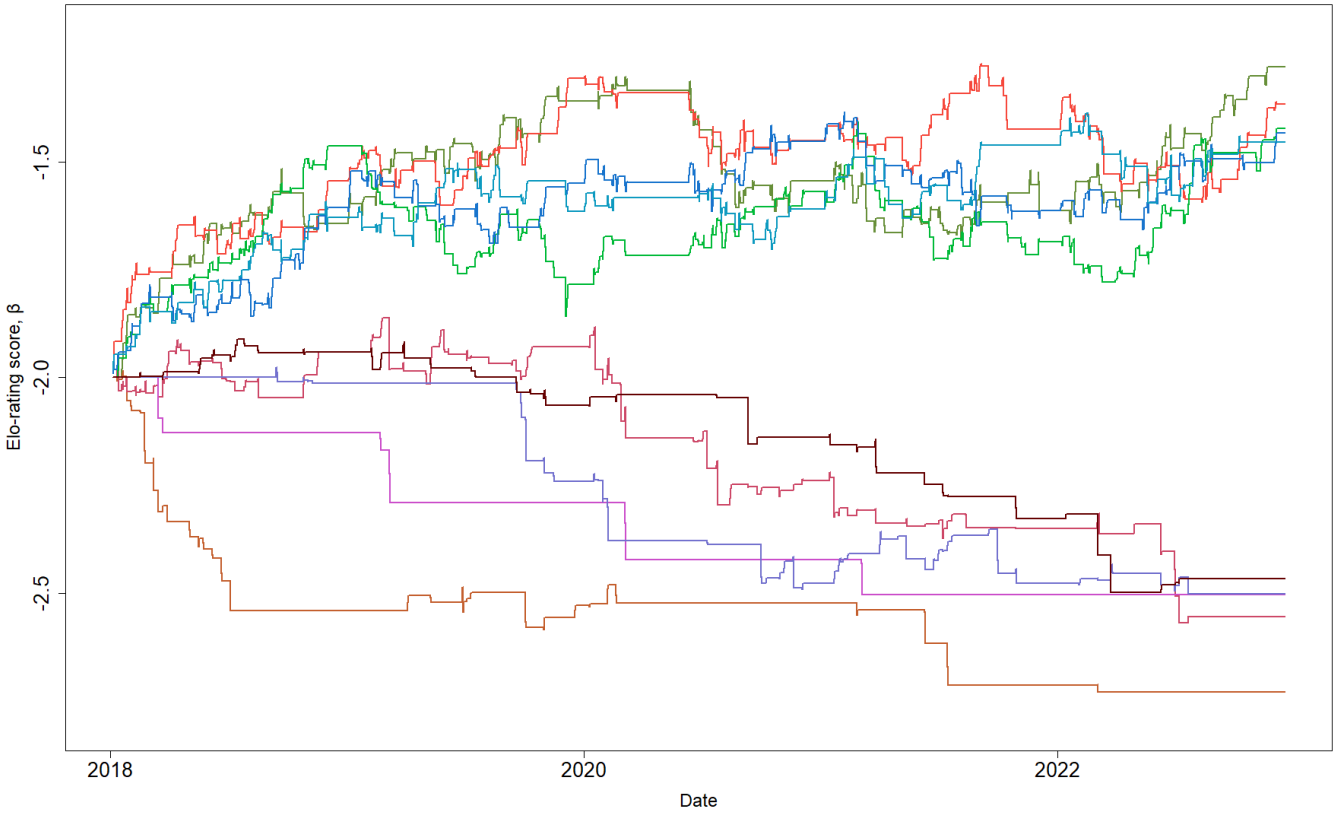


Figure 3.9: 5 best and worst Golf competitors, based on their rating scores, β

From the plot, we can see the results are similar to our previous results, in Figure 3.4, however the jumps in rating scores after updates are smaller. Again, this is due to a smaller learning rate. The best competitors seem to stay between -1.25 and -1.75 after a year of updates, whereas the worst competitors stay between -2.75 and -2.25 but take different amounts of time to reach this range, due to their inactivity compared to the better competitors who remain active.

If we look at this in more detail, we see that the better players played an average of 412 rounds, whereas the worst competitors played an average of 94. Most of the worst competitors reach the range $(-2.75, -2.25)$ in far fewer competitions compared to the better competitors.

There are also competitors who have performed well but have not been very active. They may not have received enough updates to reach their predictive rating score range or barely reach it. An example of this is Scottie Scheffler; see Figure 3.10

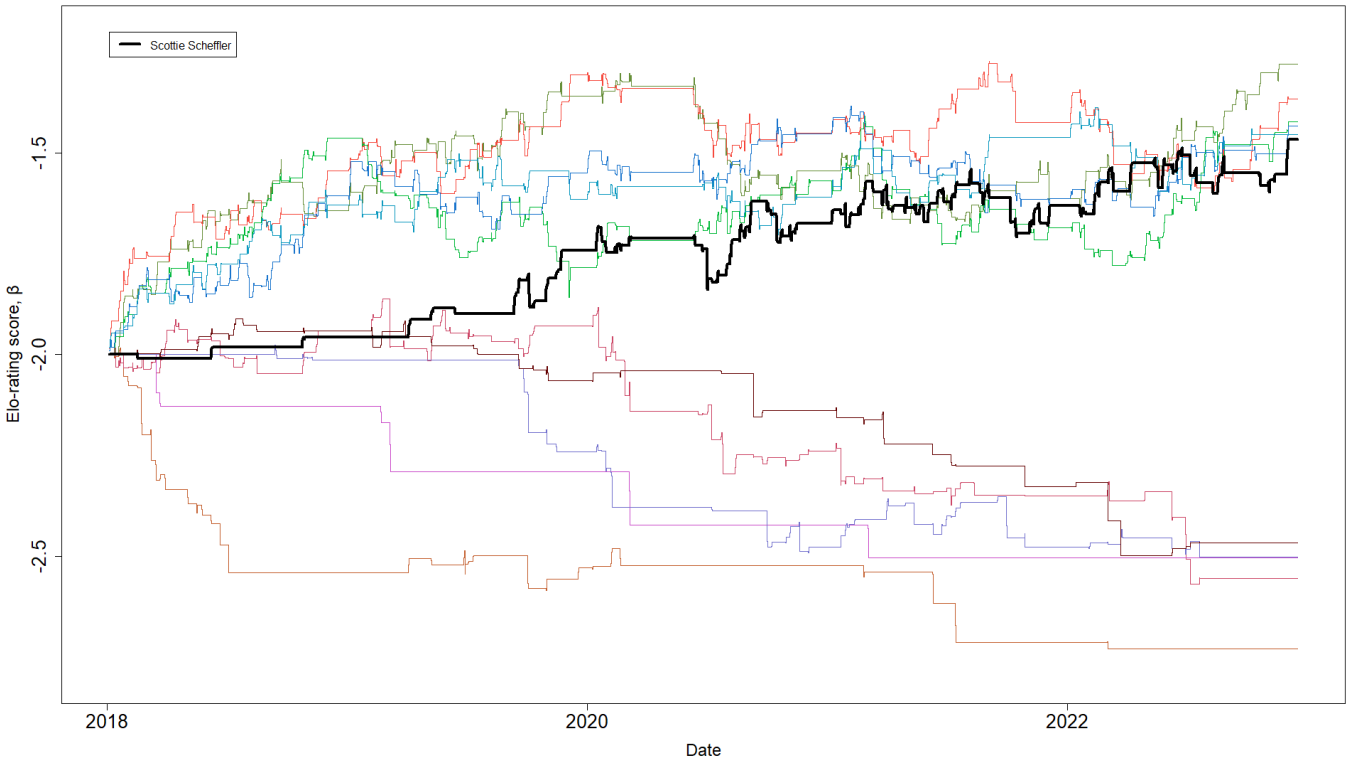


Figure 3.10: Scottie Scheffler's rating score path

We can see that he is very inactive from 2018 to 2020, competing very little throughout those years. Due to his inactivity, by 2020, when most competitors have reached their predictive rating score range, his rating is still close to -2, even though he seems to have only performed poorly in one competition. His activity begins to increase after COVID-19 and he finishes with the 6th highest rating score. However, if he had been more active in the 2018 to 2020 period, he may have been in the top 5 best competitors. In fact, at the time of writing, Scheffler is the ranked number 1 in the official world golf rankings[1] and won the US Master's in April 2024.

Chapter 4

Updating at different times

Although our initial test on the real data was successful, we can still make various improvements. In this chapter we will explore two methods of improving the accuracy of our predictions by changing when and how we update. We will use the log score to compare which of these two methods produce the most accurate predictive ratings.

4.1 Updating worst to best

We saw, in Figure 3.9, that the top 5 best competitors required more rounds played to reach their predicted rating score range compared to the bottom 5. There may be a way to fix this, or switch this around.

Recall Equation (2.5), where the probability of the event y and s occurring was split into stages, eliminating the best player in each stage. This results in smaller updates for the better players. Therefore, it would be sensible to assume that if we switch this around, so each stage eliminates the worst player, better players would receive larger updates but at the cost of the worse competitors receiving smaller updates. This would be preferred as, in general, we want to know how the better players will perform, not the worst.

To do this, we will have to reorder y and s, so that y is ordered worst to best, instead of best to worst, and s adjusted similarly. For an example, if there are $K = 5$ competitors, such that $\mathcal{K} = \{1, 2, 3, 4, 5\}$. Let $y = \{1, 2, 4, 5, 3\}$ and $s = \{1, 2, 3, 3, 4\}$ be the original list seen in Section 1.3 and $y*$ and $s*$ be the list adjusted for worst to best. Then $y* = \{3, 5, 4, 2, 1\}$ and $s* = \{1, 2, 2, 3, 4\}$, where $y*$ is ordered worst to best and $s*$ tells us their positions, where $1 \in s*$ represents last place.

This makes it simpler, instead of adjusting the GPL probability equation and deriving a new update equation, from Section 2.2, we will keep it the same. This means that, for worst to best, the lower your rating score the better you are. However, this can be confusing, so the ratings will be multiplied by -1, as they are negative. This will result in positive rating scores, where the larger the rating score the better the competitor.

After adjusting the y and s to make $y*$ and $s*$, we can then find the optimal learning rate, γ, and initial rating, β_0. Again finding the most optimal learning rate, γ, first. This gives us the following results, displayed in Figure 4.1

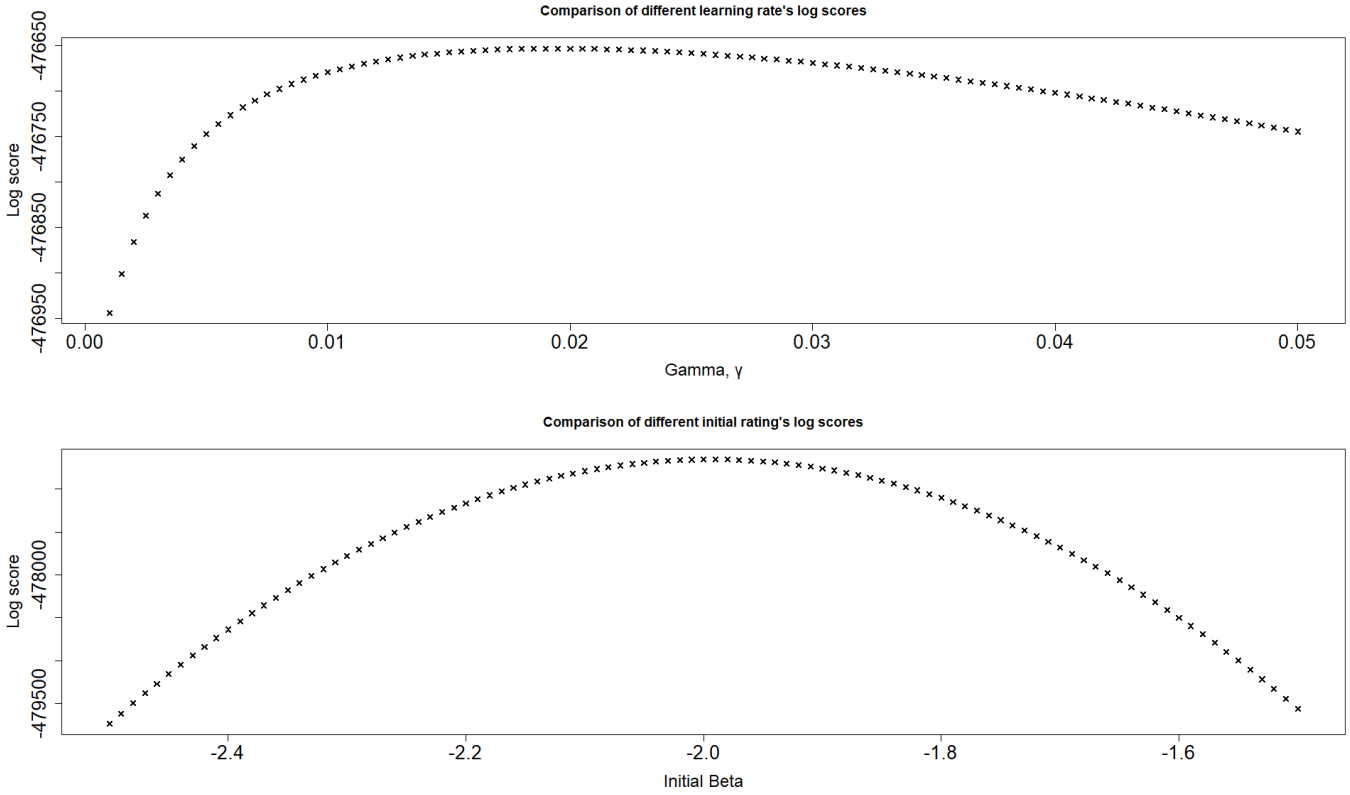


Figure 4.1: Finding the most optimal learning rate, γ, and initial rating score, β_0, when updating from worst to best

We get the largest log score, -476652.8, when $\gamma = 0.019$ and $\beta_0 = -1.99$. The plots above are very similar to Figure 3.6, but our most optimal learning rate has changed from 0.037 to 0.019, when updating from worst to best.

This log score is smaller than our previous attempt, in Section 3.4, where we got a log score of -453097.1, suggesting the predictive rating scores we obtained from worst to best are less accurate. Therefore, we will not continue to update from worst to best.

4.2 Updating after each stage

Recall Section 2.2, when calculating the probability a competitor wins a round, we split the round into stages, where the competitor that wins at stage j is positioned in j^{th} place in that round. At the moment, each competitor's rating updates after each round they compete in. Logically, if we update competitors rating score at the end of each stage, instead of each round, we should get more accurate results. Figure 4.2 shows the results obtained when finding the most optimal values for γ and β_0.

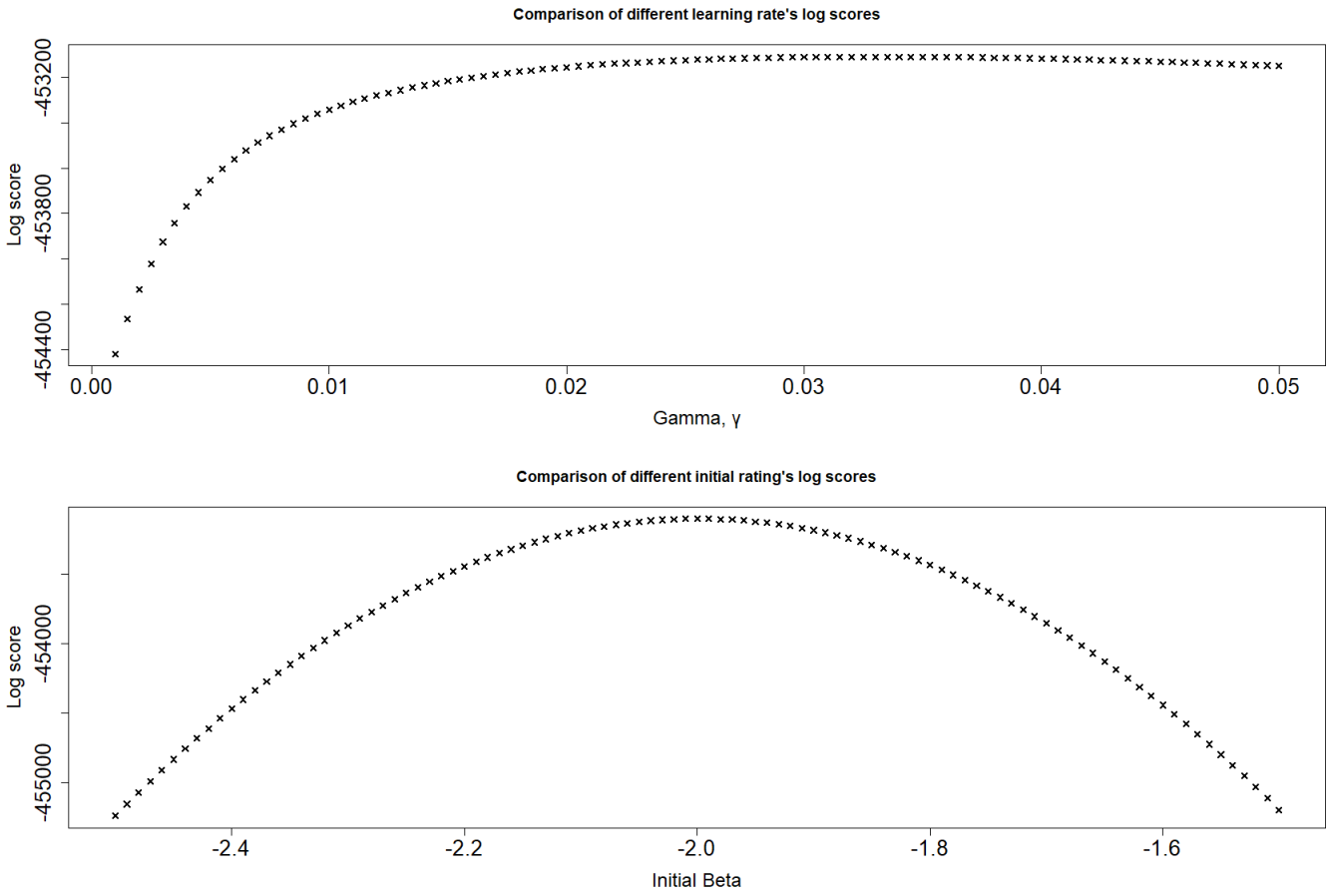


Figure 4.2: Finding the most optimal learning rate, γ, and initial rating score, β_0, when updating after every stage

Using the log score, we find the best initial rating score, $\beta_0 = -2$, and learning rate, $\gamma = 0.033$ which are very similar to our initial optimal values. With these values, the log score was -453093.6. This is slightly larger than updating after every stage but only by 3.5, which is very insignificant.

This results in the histogram and plot in Figure 4.3 and 4.4.

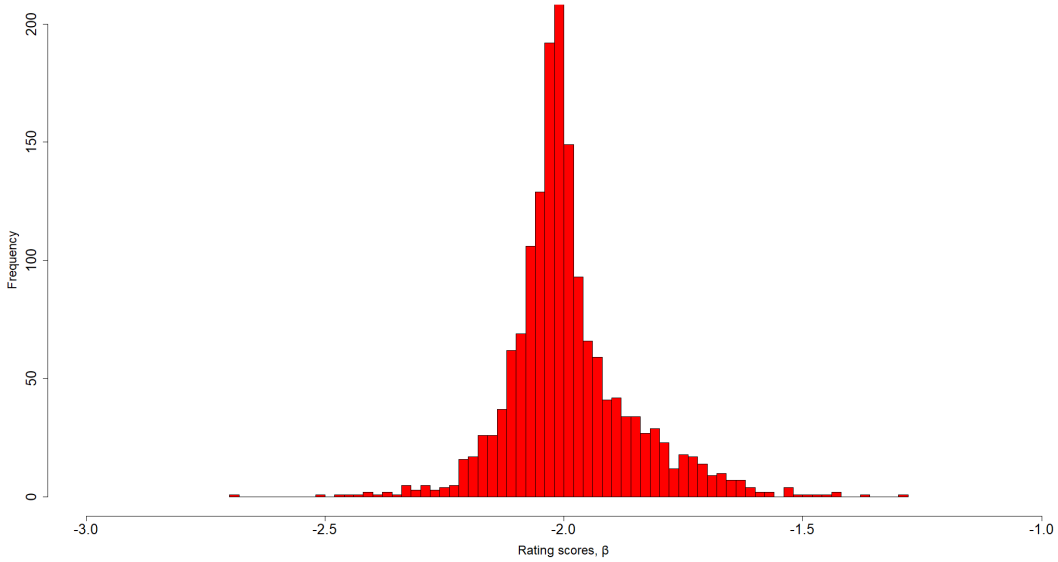


Figure 4.3: Histogram of the final rating scores, β

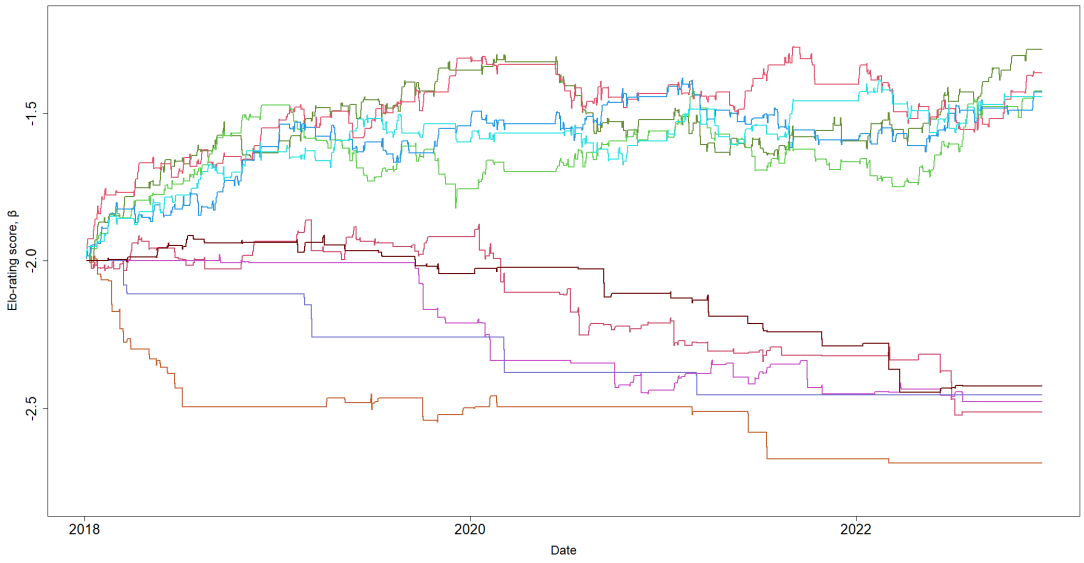


Figure 4.4: 5 best and worst Golf competitors, based on their rating scores, β

The histogram looks very similar to Figure 3.7 and the plot looks identical to Figure 3.9, and almost is. The top 5 best and worst competitors remain the same and even have very similar rating scores as we found in Section 3.4. Although the log score is very slightly larger, the difference is insignificant and it takes 60% longer to run than updating after every round. Also, when competitors compete in a round, the round is not split into stages, this idea of stages is somewhat artificial from a practical perspective, as the competitors compete simultaneously. Recall that we split rounds into stages to calculate the probability of competitors placement in that round. Therefore, it is more suitable to continue to update after every round.

Chapter 5

Competitor specific learning rates, γ_k

Recall Figure 3.5 and 3.10, where we saw Matthew Every's and Scottie Scheffler's rating paths take a longer period of time to reach their predictive rating range. One of the major reasons for this could be due to the inactivity of both competitors. The aim of this chapter is to assign a unique learning rate, γ_k, to each competitor, k, which would decrease the more they compete. We would like a competitor's learning rate to be large in their first few updates, so that they reach their predictive rating score range quicker, and decrease the more they compete, helping control the size of their updates. This will hopefully boost the ratings for less active players, such as Matthew Every and Scottie Scheffler, faster in their earlier updates. There are various methods of updating competitor specific learning rates, but we will be focusing on two.

5.1 Exponential learning rate update

The first method we will explore is an exponential method. The update equation for γ has the following form

$$\gamma_k = \gamma_{min} + (\gamma_{max} - \gamma_{min})exp(-\alpha R_k), \tag{5.1}$$

where γ_k is competitor k's learning rate, R_k is the number of rounds competitor k has competed in, γ_{min} and γ_{max} are the minimum and maximum values γ_k can be, respectively, and α is some constant.

This method seems as though it will be effective as the initial learning rate for each competitor, γ_{max} will be large. Then as a competitor competes more, their learning rate will decrease exponentially until it reaches γ_{min} where it will become fixed.

5.2 Finding the optimal values for the exponential update equation

Before we look at the results, we will find the most optimal values for β_0, γ_{min}, γ_{max} and α, using the log score. We will first find the most optimal α, when $\gamma_{min} = 0.01$ and $\gamma_{max} = 0.05$, then γ_{min} and γ_{max} together, and finally β_0.

After extensive testing of various values of β_0, γ_{min}, γ_{max} and α, we get the following results in Figure 5.1.

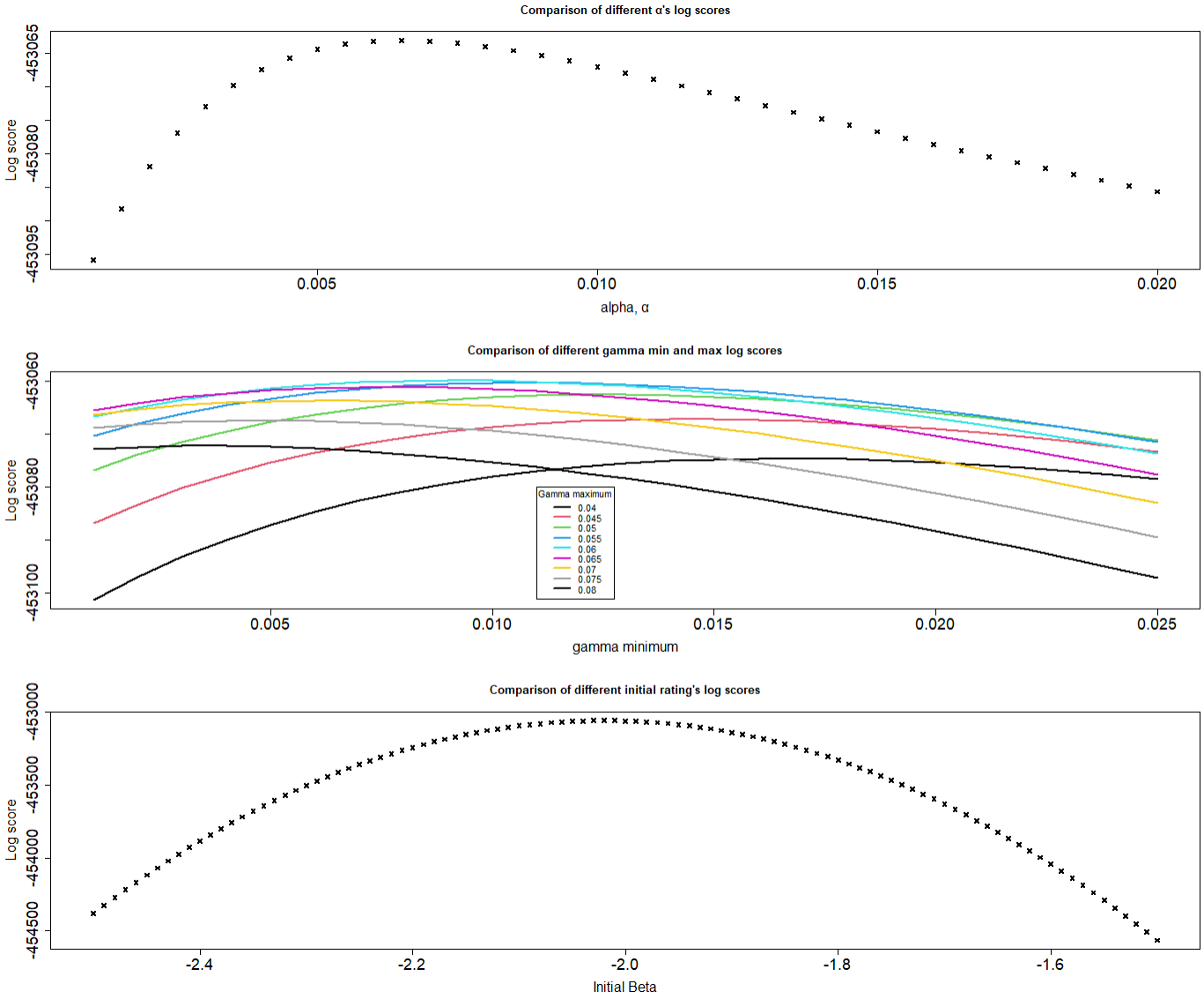


Figure 5.1: Finding the most optimal values, when updating competitor specific γs using an exponential equation

We find that the largest log score is -453057.6, when $\beta_0 = -2.02$, $\gamma_{min} = 0.009$, $\gamma_{max} = 0.06$ and $\alpha = 0.0065$. This log score is slightly larger than our original optimal updates, in Section 3.4, by only 39.5. This increase in log score is practically insignificant, however the aim was not only to make the predictive rating scores more accurate, but also help competitors reach their predictive rating scores in fewer updates.

We can see, in figure 5.2, how a competitor's learning rate, γ_k, will decrease as they compete.

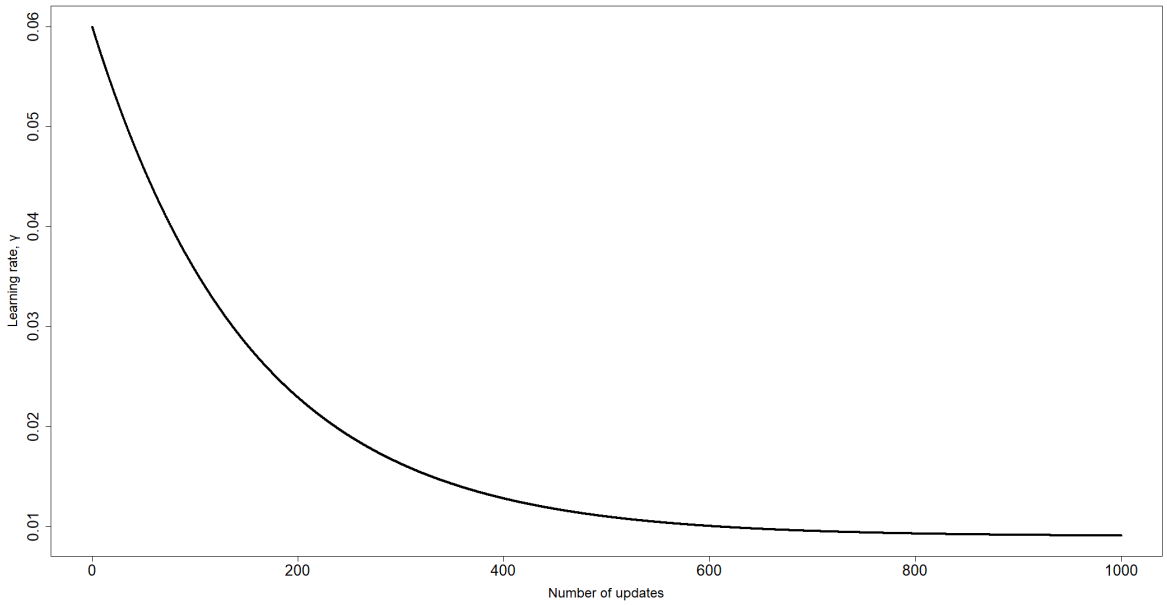


Figure 5.2: How a competitors learning rate will decrease as they compete

Figure 5.2 shows us that a competitor will receive around 900 updates, so they compete in 900 rounds, before their learning rate reaches the minimum.

5.3 Exponential learning rate update results

We obtain the results in Table 5.1 after updating using competitor specific exponential learning rate updates.

Table 5.1

Top 5		Bottom 5	
Player	β	Player	β
Jon Rahm	-1.429	Smylie Kaufman	-2.791
Rory McIlroy	-1.434	Robert Gamez	-2.705
Patrick Cantlay	-1.498	Petr Dedek	-2.616
Xander Schauffele	-1.506	David Duval	-2.590
Scottie Scheffler	-1.529	Keith Clearwater	-2.544

These results are very similar to our original optimal attempt in Section 3.4, but with some competitors who were only in the top 10 previously now being in the top 5. One competitor in particular who is now in the top 5 best competitors is Scottie Scheffler. Recall Figure 3.10, where we saw how Scottie Scheffler's inactivity during 2018 to 2020 may have resulted in his rating score being lower than it should be. He was previously placed 6th, but with the exponential update equation he is now placed 5th.

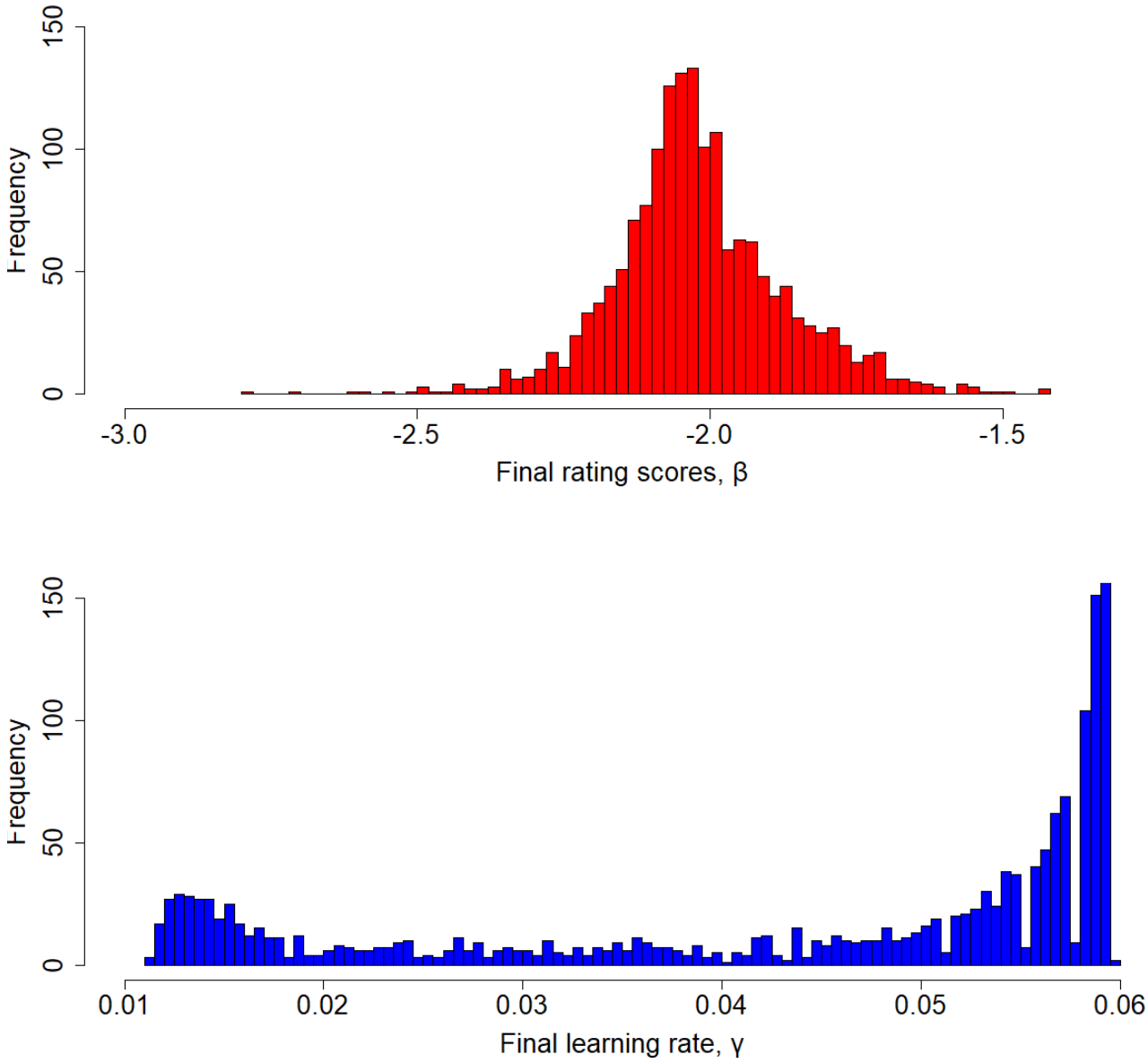


Figure 5.3: Histogram of the final rating scores, β, and final learning rates, γ

The first histogram, in Figure 5.3, shows us the final ratings are a lot less concentrated around the initial rating score, $\beta_0 = -2.02$, compared to Figure 3.7. This may be because the inactive players had larger jumps initially, helping them get away from their initial rating score, compared to before where they received too few updates to increase or decrease their rating scores by a significant amount.

The second histogram, in Figure 5.3, shows us the final learning rates for which their seems to be a large variation. A large number of competitors still have a large learning rate, with the mode being close to γ_{max} and the mean learning rate being 0.044.

Figure 5.4 shows the rating score path of the top and bottom 5 competitors. We can see how large the jumps are from the initial rating score in early 2018. Recall equation (5.1), every competitors R_k is very small in early 2018, resulting in a larger learning rate. However, after 2020, we can see that the more active competitors jumps are a lot smaller than the competitors who have not been very active.

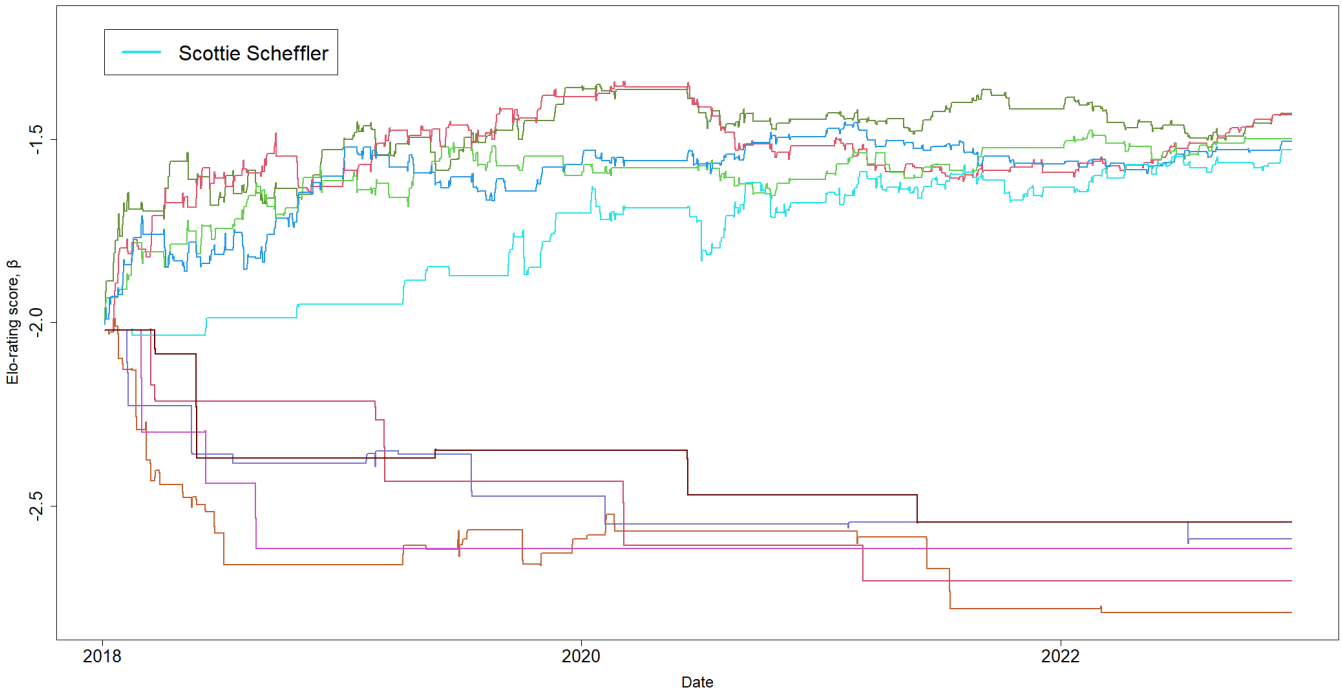


Figure 5.4: 5 best and worst Golf competitors, based on their rating scores, β

Looking at Scottie Scheffler's rating score path again, we can see he now has larger jumps in the first two years of competitions. After 2020, he becomes more active and we can see how his jumps decrease in size. Another reason he may have broke into the top 5 is because of the smaller learning rates of his competitors in the top 10. As they have competed more, their learning rates are smaller, making it harder for them to move too far from the initial rating, $\beta_0 = -2.02$.

Figure 5.5 compares his rating score path to his original path in Section 3.4, Figure 3.10. We can see how the competitor specific exponential updates help him escape the initial rating and reduce the size of his jumps later. His new path remains higher than his old path for the first two years and as his learning rate decreases, his original path from Figure 3.10 rises above his new path.

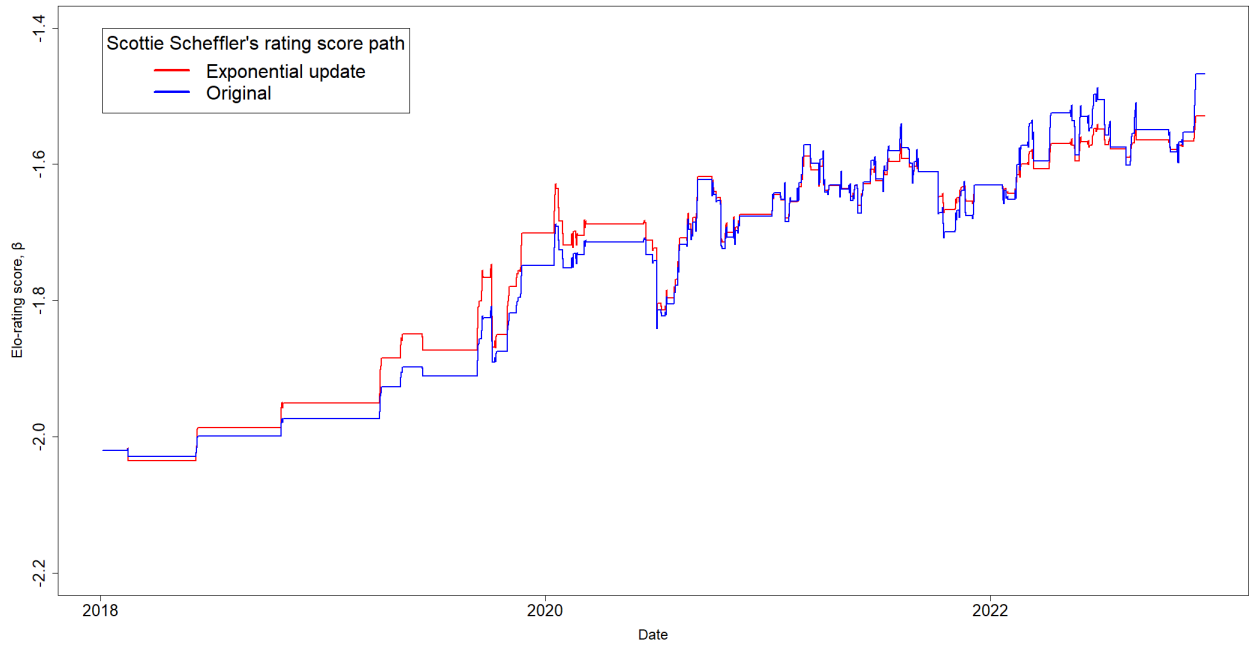


Figure 5.5: Scottie Scheffler's rating score path with and without an exponentially updated learning rate

5.4 Powell's competitor specific learning rate

The second method of updating competitor specific learning rates is based on Powell's[10] competitor specific learning rates. Powell used the following update equation for competitor's learning rate,

$$\gamma_k^{-1} \leftarrow \gamma_k^{-1} - \frac{\partial^2 \log(P^{(j)})}{\partial \beta_k^2} \tag{5.2}$$

where k is the competitor and γ_k is k's learning rate. Therefore, the first thing we must do is calculate negative second derivative of $\log(P^{(j)})$ with respect to β_k for our model.

5.5 Deriving negative second derivative of $\log(P^{(j)})$

To help calculate $\frac{\partial^2 \log(P_k^{(j)})}{\partial \beta_k^2}$, we will need the following derivatives

$$\frac{\partial \log(\theta_k)}{\partial \beta_k} = 1 - \theta_k, \qquad \frac{\partial \log(1-\theta_k)}{\partial \beta_k} = -\theta_k, \tag{5.3}$$

$$\frac{\partial \theta_k}{\partial \beta_k} = \theta_k(1-\theta_k), \qquad \frac{\partial (1-\theta_k)}{\partial \beta_k} = -\theta_k(1-\theta_k) \tag{5.4}$$

and

$$\frac{\partial P_k^{(j)}}{\partial \beta_k} = \frac{\partial}{\partial \beta_k}\left(\frac{\theta_k \prod_{i \neq \{k\}}(1-\theta_i)}{1-(1-\theta_k)\prod_{i \neq \{k\}}(1-\theta_i)}\right) = P_k^{(j)}(1-P_k^{(j)})(1-\theta_k) \tag{5.5}$$

where $P_k^{(j)}$ is the probability competitor k wins at stage j.

Recall from Equation (2.21), that $\frac{\partial log(P^{(j)})}{\partial \beta_k} = (\mathbf{I}_k^{(j)} - P_k^{(j)}) - \theta_k(1 - P_k^{(j)})$, where $\mathbf{I}_k^{(j)} = 1$ as we are assuming k wins at stage j. Then

$$\frac{\partial \log(P^{(j)})}{\partial \beta_k} = (1 - P_k^{(j)})(1 - \theta_k) \tag{5.6}$$

and

$$\begin{aligned}
\frac{\partial^2 log(P^{(j)})}{\partial \beta_k^2} &= \frac{\partial}{\partial \beta_k}(1 - P_k^{(j)})(1 - \theta_k) && (5.7)\\
&= \frac{\partial(1 - P_k^{(j)})}{\partial \beta_k}(1 - \theta_k) + (1 - P_k^{(j)})\frac{\partial(1 - \theta_k)}{\partial \beta_k} && (5.8)\\
&= -P_k^{(j)}(1 - P_k^{(j)})(1 - \theta_k)^2 - \theta(1 - P_k^{(j)})(1 - \theta_k) && (5.9)\\
&= (1 - P_k^{(j)})(1 - \theta_k)\{-\theta_k - P_k^{(j)}(1 - \theta_k)\} && (5.10)\\
&= \frac{\partial log(P^{(j)})}{\partial \beta_k}\{(0 - P_k^{(j)}) - \theta_k(1 - P_k^{(j)})\} && (5.11)\\
&= \{(1 - P_k^{(j)}) - \theta_k(1 - P_k^{(j)})\}\{(0 - P_k^{(j)}) - \theta_k(1 - P_k^{(j)})\} && (5.12)
\end{aligned}$$

If we assume competitor k loses at stage j, we get the same form. Therefore, the general equation for the second derivative is

$$\frac{\partial^2 log(P^{(j)})}{\partial \beta_k^2} = \{(1 - P_k^{(j)}) - \theta_k(1 - P_k^{(j)})\}\{(0 - P_k^{(j)}) - \theta_k(1 - P_k^{(j)})\} \tag{5.13}$$

where $k \in \mathcal{K}$ is a competitor in stage j.

5.6 Reciprocal update equation for γ_k

Using Equation 5.7, we can now form an update equation for the reciprocal of a competitor k's learning rate γ_k.

$$\gamma_k^{-1} \leftarrow \gamma_k^{-1} - \{(1 - P_k^{(j)}) - \theta_k(1 - P_k^{(j)})\}\{(0 - P_k^{(j)}) - \theta_k(1 - P_k^{(j)})\} \tag{5.14}$$

It can be seen that, no matter the outcome of a stage, the negative second derivative is always positive. This means that the more competitor k competes, the more γ_k^{-1} increases, therefore γ_k gets larger. However, the size of the negative log second derivative depends on $P_k^{(j)}$ and θ_k. Recall that $P_k^{(j)}$ is a probability, so $0 \leq P_k^{(j)} \leq 1$, and $0 \leq \theta_k \leq 1$. Also, the larger a competitors rating score, β, the larger that competitors θ is. To stop a competitor's learning rate from becoming too small, we will also add a γ_{min}, similar to the exponential update.

We can see this more clearly in Figure 5.6.

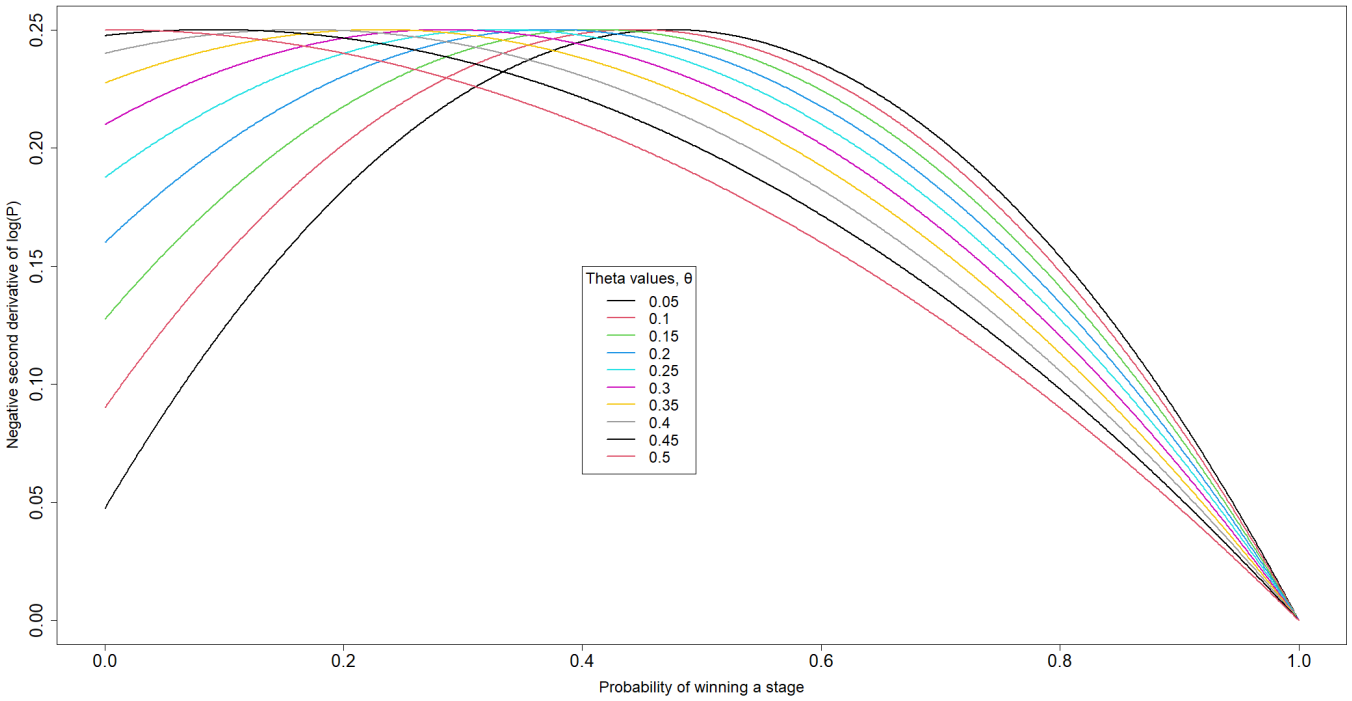


Figure 5.6: Negative second derivative against $P_k^{(j)}$, for various values of θ.

When $P_k^{(j)}$ is small or large, the negative second derivative is smaller, meaning a smaller decrease in γ. However, when $P_k^{(j)} \approx 0.4$, the change in γ is larger. This changes slightly depending on the value of a competitor's θ and we can see the maximum value for the negative second derivative is 0.25.

5.7 Finding the optimal values for the second derivative update

Now that we have derived an update equation for a competitors learning rate, we can use the log score to find the most optimal initial learning rate, γ_0, the minimum learning rate, γ_{min} and then the initial rating score, β_0. This gives the best log score so far, but only slightly better than the exponential learning rate update.

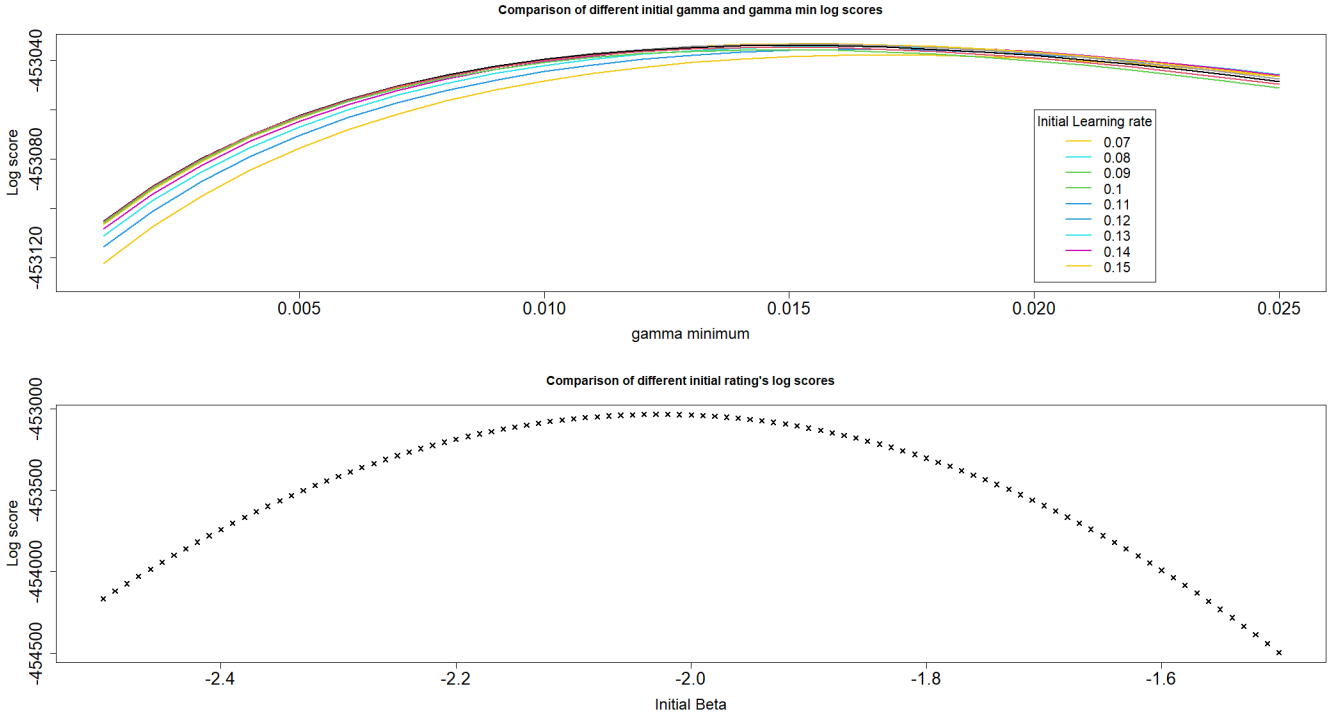


Figure 5.7: Finding the most optimal values, when updating competitor specific γs using the second derivative of P

We get the largest log score, -453034.5, when the initial learning rate is $\gamma_0 = 0.07$, the minimum learning rate is $\gamma_{min} = 0.015$ and the initial rating score is $\beta_0 = -2.03$

5.8 Log second derivative update results

This gives us the following results

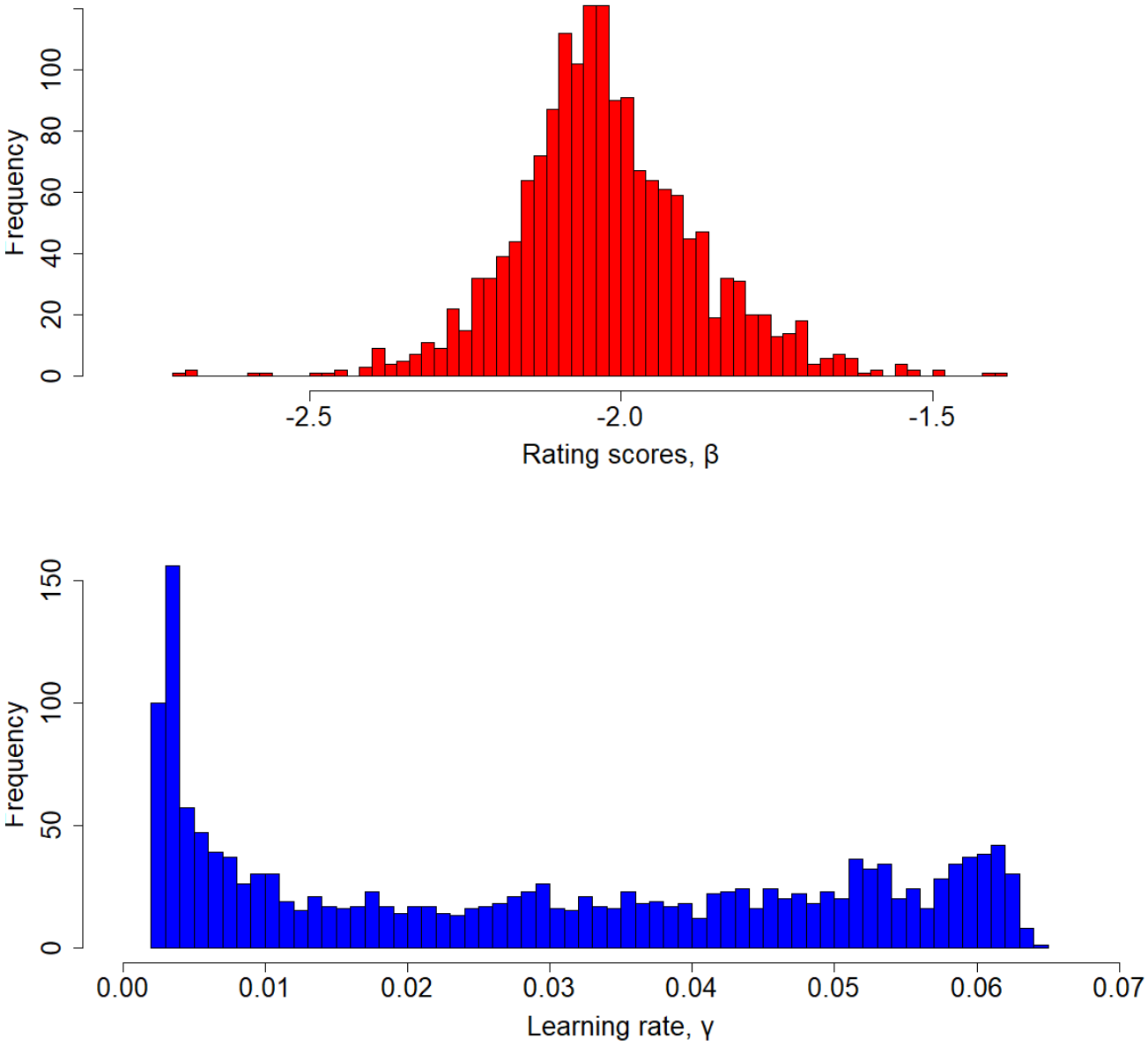


Figure 5.8: Histogram of the final rating scores, β, and final learning rates, γ

Similarly to the exponential gamma update, the first histogram, in Figure 5.8, shows us the final rating scores, which seem less concentrated than our original attempt, with many competitors rating scores moving away from the initial rating score, $\beta_0 = -2.03$. However, the second histogram, in Figure 5.8, is different. Using the second derivative update, although their is a large variation of learning rates, many competitors learning rates are close to the γ_{min} of 0.015. The mean final learning rate is 0.029.

Table 5.2 shows us the top 5 best and worst competitors which are familiar, with Scottie Scheffler making it into the top 5 with the competitor specific learning rates again.

Table 5.2

Top 5		Bottom 5	
Player	β	Player	β
Rory McIlroy	-1.391	Smylie Kaufman	-2.703
Jon Rahm	-1.419	Robert Gamez	-2.698
Patrick Cantlay	-1.489	Petr Dedek	-2.681
Xander Schauffele	-1.490	David Duval	-2.582
Scottie Scheffler	-1.521	Keith Clearwater	-2.570

Figure 5.9 shows us these competitor's rating score paths.

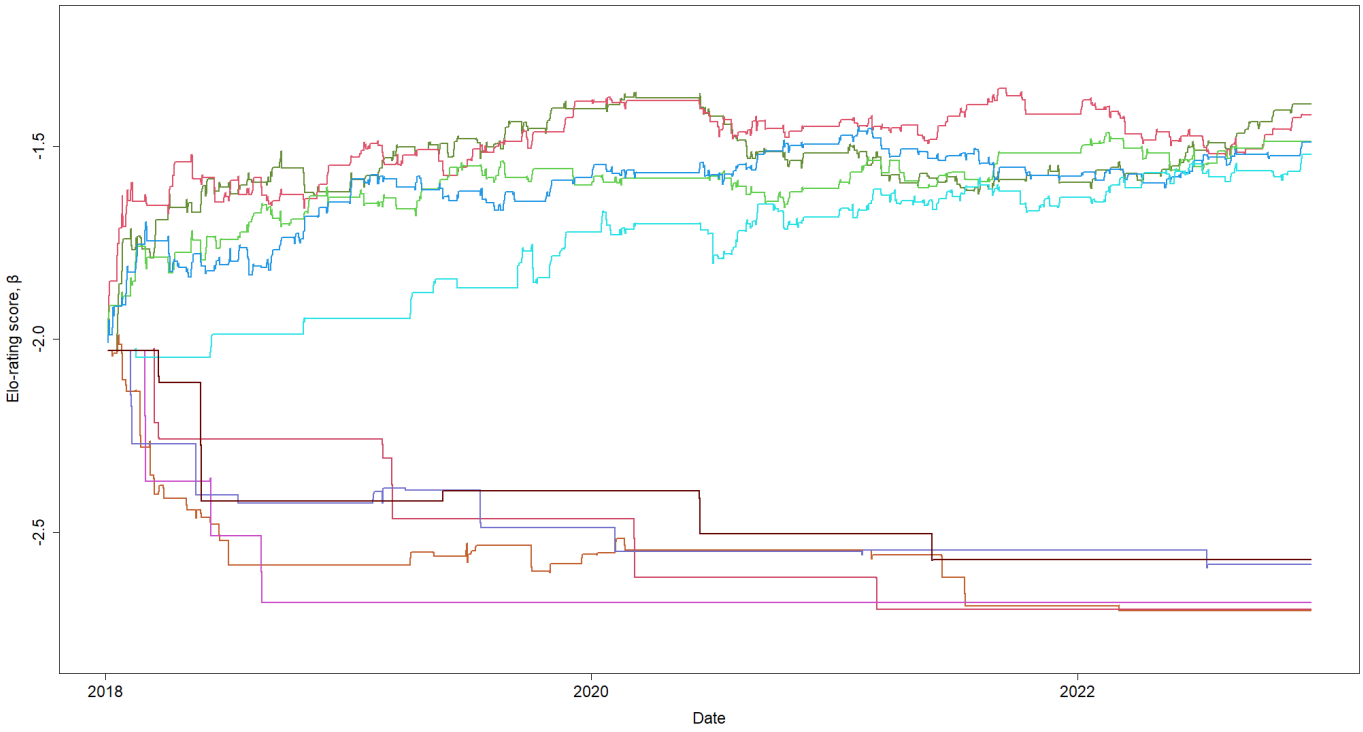


Figure 5.9: 5 best and worst Golf competitors, based on their rating scores, β

Figure 5.9 looks very similar to figure 5.4 and, overall, so do the results. We can see larger jumps in competitors initial updates which slowly decrease the more they compete.

5.9 Which competitor specific learning rate update is best for Golf?

We saw in figure 5.3 that a large proportion of competitors had similar final learning rates to their initial learning rate, $\gamma_{max} = 0.06$, when updating using the exponential method. Whereas, in figure 5.8, we saw the opposite, where most final learning rates were close to, or had reached, the minimum, $\gamma_{min} = 0.07$. The Figure 5.10 shows us why.

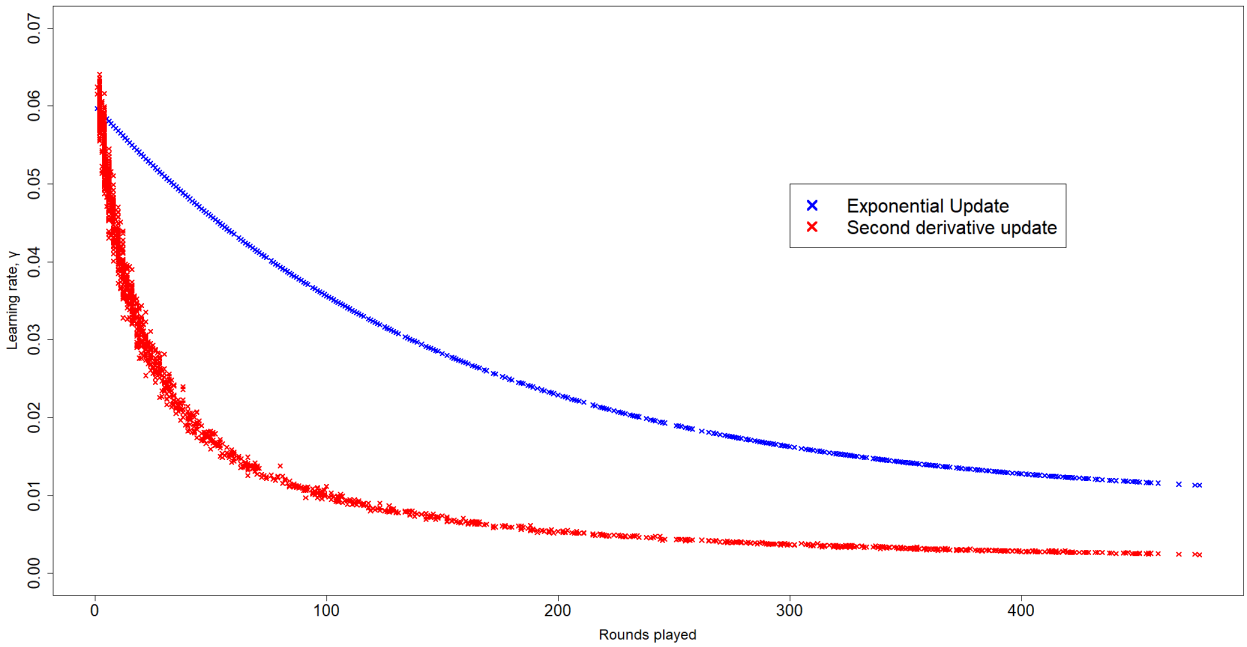


Figure 5.10: Comparison of two learning rates, γ, updates

For the exponential update, we see that a competitors learning rate will decrease slowly at a fairly consistent rate. However, when updating using the second derivative, a competitors learning rate will decrease a lot quicker and within 50 rounds a competitor will, most likely, have a learning rate smaller than 0.015, the γ_{min} for the exponential update. Note that this will not always be true, as the size of the second derivative updates are dependent on the probability of a competitor winning that round's stages.

The second derivative method has a larger log score, but only by 23.7, which is insignificant. The average rounds played in this dataset is 95, rounded to the nearest whole number. Because of this, when updating using the exponential update, most competitors have a final learning rate larger than 0.05. Comparing this to the second derivative method, where most competitors have a learning rate smaller than 0.03, the exponential update will have larger update jumps for the average competitor. For this reason as well as the log score, the second derivative update seems more suitable. However, if the dataset was larger, for an example if the average rounds played was 200, I believe an exponential update would be more suitable, as most competitors would have a much smaller final learning rate.

Chapter 6

Conclusion

6.1 Final Results

Throughout this report, we have discussed and tested various algorithms for calculating the predictive rating's of all competitors, using the log score to assess how accurate these predictive ratings are. Recall that the larger the log score, the more accurate the predictive ratings are, resulting in more accurate predictions of future competitions, using Equation (2.5). Table 6.1 is a summary of the largest log scores achieved with each method we tested throughout this project, ordered largest to smallest.

Table 6.1

Model	Log score	Difference from largest log score
2nd derivative update (Section 5.7)	-453034.5	0
Exponential update (Section 5.2)	-453057.6	-23.1
Updating after every stage (Section 4.2)	-453093.6	-59.1
Updating after every round (Section 3.4)	-453097.1	-62.6
Updating from worst to best (Section 4.1)	-476652.8	-23618.3

We can see all the methods had a similar log scores, excluding updating from worst to best, and therefore the accuracy of their predictions were similar. As a result of the second derivative update method having the largest log score, as well as the reasons discussed in Section 5.9, it is the most suitable rating system update for this data. In the second derivative update method, we updated after every round and updated the competitor specific learning rates using the second derivative method. This results in Figures 5.8 and 5.9 as well as Figure 6.1.

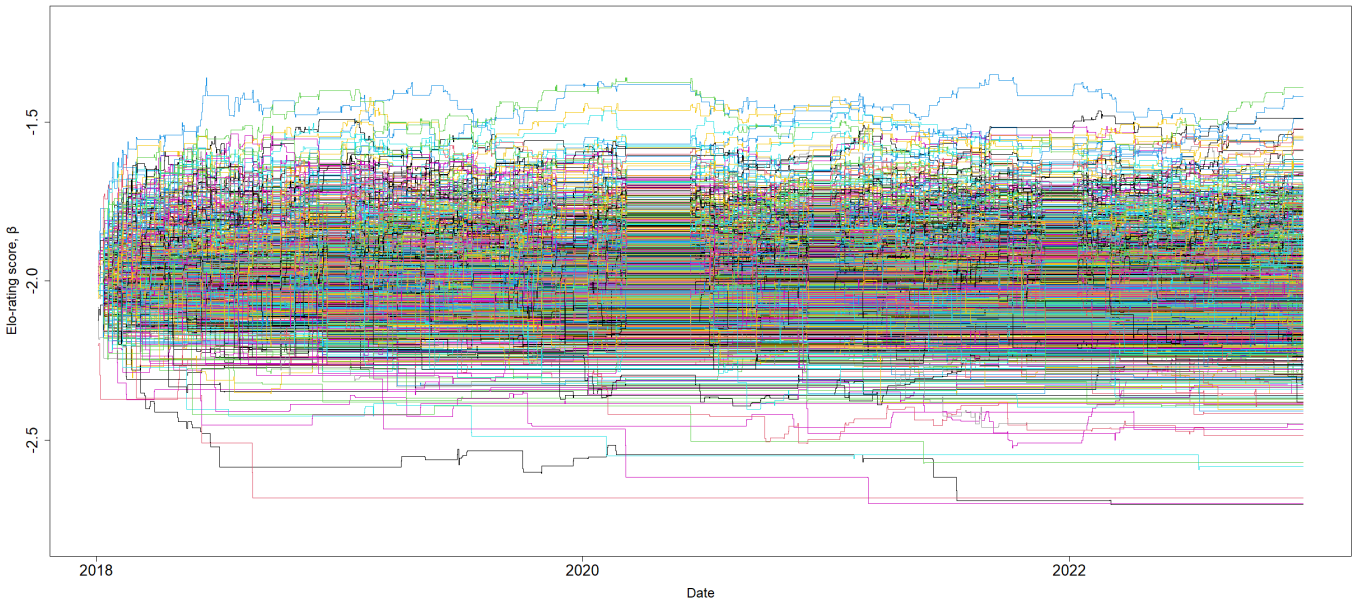


Figure 6.1: Plot of all competitors rating scores path

Table 6.2 compares the final results, from Figure 6.1, with the Official World Golf Rankings (OGWR)[1]. In particular, we will be looking at where the top 5 best competitors in this Elo-rating system were ranked in the Official World Golf Rankings in December 2022, where the dataset provided by Smartodds[2] ends.

Table 6.2

Competitor	Rating, β	Ranking according to rating, β	OGWR ranking, December 2022
Rory McIlroy	-1.391	1	1
Jon Rahm	-1.419	2	5
Patrick Cantlay	-1.489	3	4
Xander Schauffele	-1.490	4	6
Scottie Scheffler	-1.521	5	2

The results match up well with the OGWR rankings, with all competitor's ranking in the Elo rating system being within three positions of the OGWR rankings.

6.2 What Next?

There are two areas we could explore further. The first being rewarding competitors for how dominant they were, or how clearly they were the victor of a stage. At the moment, a competitor's rating score will increase by the same amount if they win by 1 point or 20. One way we could do this is by adjusting the probability equation further and deriving a new update equation for β, similar to how Equation (2.22) was derived. We could also consider a similar variable which further scales the size of an update, based on the significance of a tournament. This makes sense as competitors are more likely to try and perform at their best ability due to the importance of a big tournament.

The second is an alternative method of updating a competitor's learning rate. This method would be inspired by the learning rate update used in the Elo-rating system for chess players[4] in tournaments. Equation (6.1) is the learning rate update equation for chess players in a tournament,

$$\gamma_k = \frac{800}{N_e + m} \tag{6.1}$$

where γ_k is competitor k's learning rate, N_e is their expected number of games, m is the number of matches competitors played in that tournament. N_e is the smaller of N and N_r, where N is the number of previous games and N_r is calculated using Equation (6.2),

$$N_r = \frac{50}{\sqrt{0.6662 + 0.00000739(2569 - R_{pre})^2}} \tag{6.2}$$

where R_{pre} is the competitor's pre-tournament rating. Note that if the competitor has more than 50 previous games, then let $N = 50$. We may find adjusting the denominator most suitable, when applying to golf data, as there are a maximum of 4 rounds in a tournament in golf, where all competitors will have competed in 2 or 4 rounds. Equation (6.3) is a potential update equation for a golf competitor k's learning rate γ_k,

$$\gamma_k = \frac{A}{R_e + C} \tag{6.3}$$

where R_e is the smaller of R, the number of rounds competitor k has competed in (capped at R_{max}), and R_r, C is the number of competitions competitors played in the last t professional golf competitions and A is some constant. Using the log score, one could find the most optimal values for A, t and R_{max} as well as the optimal values for Equation (6.4),

$$R_r = \frac{R_{max}}{\sqrt{v + u(\beta_{max} - \beta_k)^2}}. \tag{6.4}$$

6.3 Acknowledgements

Finally, I would like to thank Smartodds [2] for providing the data, and Gavin Whitaker in particular, for his very helpful advice on adjusting the data.

Appendix A

Final function for rating update algorithm, in R

```
################################################
##
##All functions
##
################################################


p.j <- function(j,theta,log=TRUE)
{
  ## calculate (log) probability that each of j "wins" at stage j
  ## here j are the indices of the competitors still in at Stage j

  res <- rep(0,length(j))
  theta.i <- theta[j]

  if(length(j)==1){
    res <- 0
  }
  else{
    ell <- 0
    for(k in j){
      ell <- ell+1
      theta.i.not.k <- theta.i[-(match(k,j))]
      sum.log.1.minus.theta.i.not.k <- sum(log(1-theta.i.not.k))
      denom <- log(1-exp(log(1-theta[k])+sum.log.1.minus.theta.i.not.k))
      res.k <- log(theta[k])+sum.log.1.minus.theta.i.not.k-denom
      res[ell] <- res.k
    }
  }
  if(log!= TRUE){
    res <- exp(res)
  }
  res
}



expit <- function(x){
  exp(x)/(1+exp(x))
```

```
}



t.to.s <- function(t)
{
  ## GPL model
  ## determine s from t

  K <- dim(t)[2]+1
  n <- dim(t)[1]
  s <- matrix(0,nrow=n,ncol=K)
  for(i in 1:n){
    s[i,] <- 1+((1:K)-cumsum(c(1,t[i,])))
  }
  s
}



s.to.t <- function(s)
{

  ## GPL model
  ## determine t from s

  K <- dim(s)[2]
  n <- dim(s)[1]
  t <- matrix(0,nrow=n,ncol=K-1)
  for(i in 1:n){
    t[i,] <- ifelse(diff(s[i,])==0,1,0)
  }
  t
}



loglike.gpl <- function(y,t,theta,m.i.star)
{
  ## Generalized (geometric) Plackett-Luce (GPL)
  ## loglikelihoods over multiple datasets

  n = max(dim(y)[1],1)
  if(n == 1){
    y = t(matrix(y))
    t = t(matrix(t))
  }
  ll.vec = rep(0,n)
  for(i in 1:n){
    n.i = length(y[i,y[i,]>0])
    for(j in 1:(m.i.star[i])){
      ll.vec[i] = ll.vec[i]+log(theta[y[i,j]])-
      log(1-exp(sum(log(1-theta[y[i,j:n.i]]))))+
      (1-t[i,j])*sum(log(1-theta[y[i,(j+1):n.i]])) +
      t[i,j]*log(1-exp(sum(log(1-theta[y[i,(j+1):n.i]]))))
    }
  }
```

```
  return(ll.vec)
}



beta.update.2nd.deriv <- function(gamma,beta.0,y,s,tt,gamma.min=0.01,gamma.max
    =0.05)
{
  ## perform updates of ratings based on negative second derivative
  ## and initial ratings beta.0

  ## number of golfers
  K <- length(beta.0)

  ## number of observations
  n <- dim(y)[1]

  ## replace NA with 0 in y  for log likelihood calculation
  yy <- ifelse(is.na(y),0,y)

  ## store gamma for each golfer
  gamma.res  <- matrix(0,nrow=n,ncol=K)

  ## store results
  beta.res <- matrix(0,nrow=n,ncol=K)
  logscore <- rep(0,n)
  rounds.played <- rep(0,K)

  beta <- beta.0
  for(i in 1:n){

    ## Find players in round i and last place position
    these.players <- y[i,which(is.na(y[i,])==FALSE)]
    max.s <- max(s[i,],na.rm=TRUE)
    n.players <- length(these.players)

    theta <- expit(beta)
    ## log score of round i
    logscore[i] <- loglike.gpl(yy[i,],tt[i,],theta,n.players-1)

    P <- matrix(0,nrow=K,ncol=max.s)
    I <- matrix(NA,nrow=K,ncol=max.s)
    include <- y[i,1:n.players]

    ## counting each competitors rounds played
    competitors = rep(0,K)
    competitors[include] = 1
    rounds.played = rounds.played + competitors

    ## filling probability matrix, P, and positions matrix, I, for round i
    for(j in 1:max.s){
      P[include,j] <-  p.j(include,theta,log=FALSE)
      I[include,j] <-  0
      include <- y[i,-which(s[i,]<=j)]
      include <- include[which(is.na(include)==FALSE)]
    }
    for(j in 1:n.players){
```

```
      I[y[i,j],s[i,j]] <- 1
    }

    ## beta and gamma update
    deriv <- (I-P)-theta*(1-P)
    change <- rowSums(deriv,na.rm=TRUE)

    second.deriv <- ((I-P)-theta*(1-P))*(((1-I)-P)-theta*(1-P))
    change.gamma <- rowSums(second.deriv,na.rm=TRUE)

    inv.gamma <- (1/gamma) - change.gamma
    gamma <- 1/inv.gamma
    gamma.res[i,] <- gamma

    ##beta <- beta+gamma*change
    beta <- beta+(gamma.min+gamma)*change  ## put a minimum value in
    beta.res[i,] <- beta
  }
  lscore <- sum(logscore)
  return(list(beta.res=beta.res,logscore=logscore,
  lscore=lscore,gamma.res=gamma.res, rounds.played=rounds.played))
}
```

Bibliography

[1] https://www.owgr.com/ranking-graph?minYear=2018&maxYear=2022&player=10091&player=19195 [Accessed: (24/04/2024)].

[2] https://www.smartodds.co.uk/.

[3] Sanjana Chandan. Elo rating system – everything you need to know. 2024.

[4] Mark Glickman. Approximating Formulas for the US Chess Rating System. 2017. http://www.glicko.net/ratings/approx.pdf [Accessed: (24/04/2024)].

[5] Mark E Glickman and Albyn C Jones. Rating the chess rating system. *CHANCE-BERLIN THEN NEW YORK-*, 12:21–28, 1999.

[6] Daniel A. Henderson. Modelling and analysis of rank ordered data with ties via a generalized Plackett-Luce model. arxiv preprint arXiv:2212.08543, 2022.

[7] David R. Hunter. MM algorithms for generalized Bradley-Terry models. *The Annals of Statistics*, 32(1):384 – 406, 2004.

[8] Jan Lasek and Marek Gagolewski. Interpretable sports team rating models based on the gradient descent algorithm. *International Journal of Forecasting*, 37(3):1061–1071, 2021.

[9] John I. Marden. *Analyzing and modeling rank data*. CRC Press, 1995.

[10] Ben Powell. Generalizing the Elo rating system for multiplayer games and races: why endurance is better than speed. *Journal of Quantitative Analysis in Sports*, 19(3):223–243, 2023.